Calculating π on the 6809 SBC

From TSC Extended BASIC to a native FLEX command — five versions of Machin's formula, and what each change bought

Version	What runs where	1,000 digits
1	Everything in BASIC	≈ 17 h (est.)
2	BASIC + machine-code divide	≈ 7.4 h (est.)
3	BASIC + machine-code divide and add/subtract	7 min 45 s
4	BASIC + fast 16-bit divide path	6 min 00 s
5	**PI.CMD — all machine code under FLEX**	**2 min 20 s**

Phil Macphail · 6809 single-board computer, 2 MHz bus · FLEX 9 · TSC Extended BASIC (XB5) · A09 assembler

October 2026

Contents

1 Background . . . 3

2 The method . . . 3

- Machin’s formula . . . 3
- Refinements carried over from the Lynx original . . . 3
- Changes made in the reconstruction . . . 3

3 The hardware . . . 4

- Two findings from getting the program running . . . 4
- Calling convention for machine code . . . 4

4 The five versions . . . 5

- Version 1 — pure BASIC . . . 5
- Version 2 — machine-code divide . . . 5
- Version 3 — machine-code add/subtract . . . 5
- Version 4 — fast divide path . . . 6
- Version 5 — PI.CMD, a native FLEX command . . . 6

5 Performance summary . . . 6

- Where the time went . . . 7

Appendix A — Version 1: pure BASIC . . . 8

- PI_MACHIN.BAS — version 1 . . . 8

Appendix B — Version 2: machine-code divide . . . 11

- PI_MACHIN.BAS — version 2 . . . 11
- PIDIV.ASM . . . 14

Appendix C — Version 3: machine-code divide and add/subtract . . . 17

- PI_MACHIN.BAS — version 3 . . . 17
- PIMATH.ASM — version 3 . . . 20

Appendix D — Version 4: fast divide path . . . 25

- PI_MACHIN.BAS — version 4 . . . 25
- PIMATH.ASM — version 4 . . . 28

Appendix E — Version 5: PI.CMD . . . 34

- PI.ASM . . . 34

1 Background

The starting point was a BASIC program written years ago on a Computers Lynx to calculate π to an arbitrary number of digits. The Lynx had paged memory, and the program used it to hold a number far longer than BASIC's own arrays would allow, one decimal digit per byte, read and written with PEEK and POKE. With TSC Extended BASIC now running under FLEX on the 6809 single-board computer, which also has paged memory, the obvious thing to do was to rebuild it.

This document follows that program through five versions. Version 1 is a faithful pure-BASIC reconstruction of the original method. Versions 2 to 4 move the inner loops into 6809 machine code one routine at a time, timing each step so the effect of every change can be seen on its own. Version 5 drops BASIC entirely and becomes a FLEX command, PI.CMD. The algorithm itself never changes; only the engine underneath it does.

Every version was first run in an emulator that models the paging hardware, and checked against a reference value of π to 1,000 places, before being run on the real board. All the listings are in the appendices.

2 The method

Machin's formula

The program uses John Machin's 1706 formula, π/4 = 4·arctan(1/5) − arctan(1/239), with each arctangent expanded as the series

```
arctan(1/x) = 1/x - 1/(3x^3) + 1/(5x^5) - 1/(7x^7) + ...
```

In multi-precision terms the work reduces to three operations on a long number: dividing it by a small integer, adding it to another, and subtracting it from another. Apart from the final ×4 steps no long multiplication is needed. The 1/5 series gains about 1.4 decimal digits per term and the 1/239 series about 4.8, so for n digits the 1/5 series needs about n/1.4 terms and does almost all the work.

Refinements carried over from the Lynx original

- **One decimal digit per byte** in paged RAM, accessed with PEEK and POKE. π/4 is less than 1, so only the fractional digits are stored and the leading 3 appears as the carry out of the final ×4.
- **Precomputed 1/239.** Since 239 × 4649 = 1,111,111, the decimal expansion of 1/239 repeats with a period of only seven digits: 0.0041841 0041841 … The starting value is tiled from that pattern instead of being divided out.
- **Guard digits.** Each term truncates its last digit, so errors of about one unit accumulate per term. Six extra digits are computed and not printed; that is enough for tens of thousands of terms.

Changes made in the reconstruction

- **Divide instead of multiply.** The Lynx version held $1/x^2$ as a long decimal and formed each new power by long multiplication, roughly n^2 operations per term. Dividing the previous power by the small integer x^2 (25 or 57121) does the same job in a single pass of short division, about n operations, and needs no extra buffer. The same divide routine also forms each term, P/(2k+1). It works because both divisors are small: the running remainder never exceeds 571,219.

- **Skipping leading zeros.** Each power term is smaller than the last, so its leading zeros grow steadily. The program records the index F of the first non-zero digit and starts every pass there, roughly halving the work.
- **Combined add/subtract.** One routine adds or subtracts a buffer, selected by a sign flag, and propagates the carry or borrow above the starting digit.

3 The hardware

The SBC has 512 KB of RAM, seen as 128 blocks of 4 KB. The first 16 blocks are the conventional 64 KB address space. A 4 KB window can be pointed at any block by writing the block number to a page register.

Item	Detail
Page register	$F280 (62080), 74HC373 latch qualified with E. Intended as write-only.
Window	$E000–$EFFF (57344–61439), decoded from Y6 of a 74HC138 on A12–A14, enabled by A15
Native block	Block 14 is the RAM normally at $E000. It is mapped back in before any call to FLEX.
FLEX	Standard layout: DOS at $C000–$DFFF, MEMEND = $BFFF, utility command area $C100–$C6FF
Bus clock	2 MHz (all measurements in this document)

Two findings from getting the program running

- **The window is at $E000, not $D000.** The first runs printed 0.0000000000. A write/read-back test across all 112 spare blocks failed on 111 of them, and a direct test showed that block selection had no effect. Tracing the decoder showed the window select on Y6, which is $E000–$EFFF. Until then every “paged” access had gone to ordinary RAM at $D000, which is in FLEX’s own memory. With the window address corrected the test passed and the program worked.
- **The page register also latches on reads.** It was designed on the assumption it would never be read, so a read clocks whatever is on the data bus into it and the window jumps to a random block. The monitor’s memory command hung on $F280 for this reason, since it reads before writing. Programs must only ever write $F280, which also means keeping a copy of the current block in a variable.

Calling convention for machine code

TSC Extended BASIC’s USR function passes one 16-bit value at MEMEND−4 and jumps to the address held at MEMEND−2. With MEMEND = $BFFF the argument is at $BFFB and the vector at $BFFD. The routine must return with RTS and the stack as it found it. Because only one value passes each way, versions 2 to 4 use a parameter block at $C100, filled with DPOKE, and select a routine by changing the vector.

4 The five versions

Version 1 — pure BASIC

A direct reconstruction of the method above. Four buffers of 28 blocks each (114,688 digits) hold the power term P, the current term T and the two arctangent accumulators. Every digit access goes through two subroutines that work out the block and offset, write the page register only when the block changes, and PEEK or POKE through the window. Block 14 is mapped back in before every PRINT and INPUT.

Lines	Routine
7000–7290	Arctangent series: term, accumulate, next power, progress line every 10 terms
8000–8220	Divide a buffer by V into another, returning the first non-zero digit F1
8300–8530	Add or subtract one buffer into another, with carry/borrow propagation
8600–8860	Normalise a digit, multiply by a small integer, clear a buffer
9000–9230	Get digit, put digit, restore native block

Performance: 100 digits in 12 min 02 s. The board ran about 610 BASIC statements per second, and emulator statement counts at that rate predicted every later BASIC timing to within a few percent. 1,000 digits needs about 38 million statements: roughly 17 hours.

Version 2 — machine-code divide

The divide is called twice per term (once to form the term, once for the next power), so it was moved first. PIDIV.ASM, at $C110, does the same short division with a 24-bit remainder held in memory: the remainder is multiplied by 10 with shifts and adds, the next digit is added, and the quotient digit is found by up to nine trial subtractions of V. It maps blocks itself, never reads $F280, and leaves block 14 mapped on return.

In BASIC the subroutine at line 8000 shrinks to a calling sequence:

```
8030 DPOKE P1,Q2          source buffer
8040 DPOKE P1+2,Q3        destination buffer
8050 DPOKE P1+4,N         digit count
8060 DPOKE P1+6,F         first digit to process
8070 DPOKE P1+8,V         divisor
8080 DPOKE U0,D1          USR vector -> $C110
8090 X=USR(0)
8100 F1=PEEK(P1+12)*256+PEEK(P1+13)
8110 C=14                 routine leaves block 14 mapped
```

The 285 bytes of machine code are POKEd in at start-up from DATA lines, with a checksum so that a corrupted transfer stops with a message rather than crashing. **Performance:** 100 digits in 4 min 36 s, 2.6× faster. About 530 machine cycles per digit against roughly 25 ms per digit in BASIC.

Version 3 — machine-code add/subtract

With the divide gone, nearly all the remaining time was the add/subtract loop, still in BASIC. PIMATH.ASM keeps the divide unchanged at $C110 and adds ADDSUB at $C240. It works from digit N−1 down to F, then carries or borrows into the digits above F for as long as needed. The sign is passed as 0 (add) or 1 (subtract), computed as (1−S)/2, so no negative value has to go through DPOKE. About 120 cycles per digit.

Performance: 100 digits in 25 s, 29× faster than version 1; 1,000 digits in 7 min 45 s. At this point the BASIC that remains (loading the code, DPOKEs before each of about 2,800

calls, printing) is a large share of the run.

Version 4 — fast divide path

The divide's quotient loop was now the hot spot. When V ≤ 6553, the value R×10+digit can never exceed 65,535, so the whole step fits in 16 bits. The new fast path keeps the remainder in the D register, counts the quotient in U, and first tries subtracting 5V so that it then needs at most four or five single subtractions instead of up to nine. That covers ÷25 and every (2k+1) divisor for runs below about 9,000 digits; ÷57121 still takes the original 24-bit path. ADDSUB moved to $C300 to make room.

```
FL1     SUBD    V5          try 5V first
        BCC     FL2
        ADDD    V5          less than 5V - undo, Q starts at 0
        LDU     #0
        BRA     FL3
FL2     LDU     #5          at least 5V - Q starts at 5
FL3     SUBD    VV          then single subtractions
        BCS     FL4
        LEAU    1,U
        BRA     FL3
FL4     ADDD    VV          D = remainder
```

÷25 fell from 532 to 212 cycles per digit. **Performance:** 1,000 digits in 6 min 00 s. At 100 digits the gain is invisible, because the run is now dominated by BASIC: the loader alone POKEs 749 bytes, about 7 seconds.

Version 5 — PI.CMD, a native FLEX command

The last step removes BASIC. PI.ASM reuses the divide and add/subtract code as it stands and adds the parts BASIC used to do: number input, clearing and filling buffers, the ×4 steps (using the 6809's MUL instruction), the arctangent series loop and printing. Usage is `PI 1000`, or just `PI` to be prompted; it accepts 1 to 65,529 digits.

FLEX entry	Address	Use
WARMS	$CD03	Return to FLEX
PUTCHR	$CD18	Print one character
INBUFF	$CD1B	Read a line into the line buffer
PSTRNG	$CD1E	Print a string, preceded by CR/LF
PCRLF	$CD24	New line
Line buffer pointer	$CC14	Finds the argument after the command name

Each buffer gets 16 blocks (65,536 digits), at blocks 16, 32, 48 and 64. Number parsing is done by the program itself rather than with FLEX's INDEC and OUTDEC, and block 14 is restored before every FLEX call. In the emulator the FLEX routines deliberately corrupted A, B and X to check that nothing relied on them being preserved. **Performance:** 1,000 digits in 2 min 20 s, all of it arithmetic: about 280 million machine cycles. 100 digits takes about 2 seconds.

5 Performance summary

All times are on the 2 MHz board. Measured times are from runs on the hardware; estimates use the emulator's statement and cycle counts at 610 BASIC statements per second plus 2 million machine cycles per second, a model that matched each measured run to within a few percent.

Version	Change	100 digits	Speed-up	1,000 digits	Speed-up
1	Pure BASIC	12:02	1×	≈ 17.4 h (est.)	1×
2	+ machine-code divide	4:36	2.6×	≈ 7.4 h (est.)	≈ 2.4×
3	+ machine-code add/subtract	0:25	29×	7:45	135×
4	+ fast 16-bit divide path	≈ 0:25 (est.)	≈ 29×	6:00	174×
5	**PI.CMD, all machine code**	**≈ 0:02 (est.)**	**≈ 360×**	**2:20**	**≈ 450×**

Where the time went

- **Find the hot spot before optimising.** Moving the divide first gave 2.6×; moving add/subtract next gave a further 11×, because once the divide was fast the BASIC add/subtract loop was almost all that was left.
- **Interpreter overhead hides machine-code gains.** The fast divide path halved the arithmetic but made no visible difference at 100 digits, where loading the code and making 295 USR calls cost more than the calculation. It only shows at 1,000 digits.
- **The work grows with the square of the digit count.** Ten times the digits means about a hundred times the work, so PI.CMD should take around four hours for 10,000 digits.

Appendix A — Version 1: pure BASIC

Files: PI_MACHIN.BAS (version 1)

Performance and layout: 100 digits: 12 min 02 s (measured). 1,000 digits: ≈ 17.4 h (estimated). Run under TSC Extended BASIC.

PI_MACHIN.BAS — version 1

```
100 REM PI BY MACHIN'S FORMULA - PURE BASIC VERSION
110 REM PI/4 = 4*ARCTAN(1/5) - ARCTAN(1/239)
120 REM ONE DECIMAL DIGIT PER BYTE IN PAGED RAM
130 REM SELECT REGISTER $F280, 4K WINDOW AT $E000-$EFFF
140 REM BLOCK 14 IS THE NATIVE $E000 RAM. MAP IT BACK
150 REM IN (GOSUB 9200) BEFORE EVERY PRINT OR INPUT.
160 R0=62080
170 W0=57344
180 C=-1
190 GOSUB 9200
200 REM BUFFERS: 28 BLOCKS (114688 DIGITS) EACH
210 P0=16
220 T0=44
230 A0=72
240 B0=100
250 DIM P9(6)
260 FOR I=0 TO 6
270 READ P9(I)
280 NEXT I
290 DATA 0,0,4,1,8,4,1
300 INPUT "NUMBER OF DIGITS";N1
310 N=N1+6
320 IF N<=114688 THEN 350
330 PRINT "TOO MANY DIGITS"
340 GOTO 300
350 REM ---- ARCTAN(1/5), STARTING FROM 1/5 = 0.2
360 PRINT "ARCTAN(1/5)"
370 Q3=A0
380 GOSUB 8800
390 Q3=P0
400 GOSUB 8800
410 Q=P0
420 I=0
430 D=2
440 GOSUB 9100
450 X2=25
460 Z=A0
470 GOSUB 7000
480 REM ---- ARCTAN(1/239), 1/239 = 0.0041841 REPEATING
490 GOSUB 9200
500 PRINT "ARCTAN(1/239)"
510 Q3=B0
520 GOSUB 8800
530 Q=P0
540 FOR I=0 TO N-1
550 D=P9(I-7*INT(I/7))
560 GOSUB 9100
570 NEXT I
580 X2=57121
590 Z=B0
600 GOSUB 7000
610 REM ---- PI = 4*(4*ARCTAN(1/5) - ARCTAN(1/239))
620 Q3=A0
630 V=4
640 GOSUB 8700
650 Q2=B0
660 Q3=A0
670 S=-1
680 F=0
690 GOSUB 8300
700 Q3=A0
710 V=4
720 GOSUB 8700
730 I0=E
740 REM ---- PRINT, 50 DIGITS PER LINE
750 GOSUB 9200
760 PRINT
```

```
770 PRINT CHR$(48+I0);".";
780 L$=""
790 FOR I=0 TO N1-1
800 Q=A0
810 GOSUB 9000
820 L$=L$+CHR$(48+D)
830 IF LEN(L$)=50 THEN 860
840 IF I=N1-1 THEN 860
850 GOTO 890
860 GOSUB 9200
862 IF I<50 THEN 870
864 PRINT "  ";
870 PRINT L$
880 L$=""
890 NEXT I
900 GOSUB 9200
910 END
7000 REM ---- ARCTAN SERIES
7010 REM BUFFER P0 HOLDS 1/X, X2=X*X, RESULT ADDED INTO BUFFER Z
7020 REM F = INDEX OF FIRST NON-ZERO DIGIT OF P (SKIPS LEADING ZEROS)
7030 M=0
7040 S=1
7050 F=0
7060 IF F>=N THEN 7290
7070 REM T = P/(2M+1)
7080 Q2=P0
7090 Q3=T0
7100 V=2*M+1
7110 GOSUB 8000
7120 REM Z = Z + S*T
7130 Q2=T0
7140 Q3=Z
7150 GOSUB 8300
7160 REM P = P/X2, IN PLACE
7170 Q2=P0
7180 Q3=P0
7190 V=X2
7200 GOSUB 8000
7210 F=F1
7220 M=M+1
7230 S=-S
7240 IF INT(M/10)*10<>M THEN 7060
7250 GOSUB 9200
7260 PRINT "TERM";M;"  LEADING ZEROS";F
7270 GOTO 7060
7290 RETURN
8000 REM ---- DIVIDE BUFFER Q2 BY V INTO BUFFER Q3, DIGITS F TO N-1
8010 REM F1 RETURNS THE FIRST NON-ZERO QUOTIENT DIGIT (N IF NONE)
8020 R=0
8030 F1=N
8040 FOR I=F TO N-1
8050 Q=Q2
8060 GOSUB 9000
8070 R=R*10+D
8080 D=INT(R/V)
8090 R=R-D*V
8100 IF R>=0 THEN 8130
8110 D=D-1
8120 R=R+V
8130 IF R<V THEN 8160
8140 D=D+1
8150 R=R-V
8160 IF D=0 THEN 8190
8170 IF F1<N THEN 8190
8180 F1=I
8190 Q=Q3
8200 GOSUB 9100
8210 NEXT I
8220 RETURN
8300 REM ---- ADD S*(BUFFER Q2) INTO BUFFER Q3, DIGITS N-1 DOWN TO F
8310 REM S=1 ADDS, S=-1 SUBTRACTS
8320 E=0
8330 FOR I=N-1 TO F STEP -1
8340 Q=Q2
8350 GOSUB 9000
8360 W1=D
8370 Q=Q3
8380 GOSUB 9000
8390 D=D+S*W1+E
```

```
8400 GOSUB 8600
8410 GOSUB 9100
8420 NEXT I
8430 REM CARRY OR BORROW INTO DIGITS ABOVE F
8440 I=F-1
8450 IF E=0 THEN 8530
8460 IF I<0 THEN 8530
8470 GOSUB 9000
8480 D=D+E
8490 GOSUB 8600
8500 GOSUB 9100
8510 I=I-1
8520 GOTO 8450
8530 RETURN
8600 REM ---- NORMALISE DIGIT D, E RETURNS CARRY (+1) OR BORROW (-1)
8610 E=0
8620 IF D>=0 THEN 8650
8630 D=D+10
8640 E=-1
8650 IF D<10 THEN 8680
8660 D=D-10
8670 E=1
8680 RETURN
8700 REM ---- MULTIPLY BUFFER Q3 BY SMALL INTEGER V, IN PLACE
8710 REM E RETURNS THE CARRY OUT OF THE TOP (INTEGER PART)
8720 E=0
8730 Q=Q3
8740 FOR I=N-1 TO 0 STEP -1
8750 GOSUB 9000
8760 D=D*V+E
8770 E=INT(D/10)
8780 D=D-E*10
8790 GOSUB 9100
8795 NEXT I
8798 RETURN
8800 REM ---- CLEAR BUFFER Q3 TO ZERO
8810 Q=Q3
8820 D=0
8830 FOR I=0 TO N-1
8840 GOSUB 9100
8850 NEXT I
8860 RETURN
9000 REM ---- GET DIGIT I OF BUFFER Q INTO D
9010 K=Q+INT(I/4096)
9020 IF K=C THEN 9050
9030 POKE R0,K
9040 C=K
9050 D=PEEK(W0+I-(K-Q)*4096)
9060 RETURN
9100 REM ---- PUT D INTO DIGIT I OF BUFFER Q
9110 K=Q+INT(I/4096)
9120 IF K=C THEN 9150
9130 POKE R0,K
9140 C=K
9150 POKE W0+I-(K-Q)*4096,D
9160 RETURN
9200 REM ---- MAP NATIVE BLOCK 14 BACK IN BEFORE PRINT/INPUT
9210 POKE R0,14
9220 C=14
9230 RETURN
```

Appendix B — Version 2: machine-code divide

Files: PI_MACHIN.BAS (version 2) and PIDIV.ASM

Performance and layout: 100 digits: 4 min 36 s (measured), 2.6× version 1. Machine code at $C110–$C22C, loaded from DATA lines (285 bytes, checksum 37202).

PI_MACHIN.BAS — version 2

```
100 REM PI BY MACHIN'S FORMULA - PURE BASIC VERSION
110 REM PI/4 = 4*ARCTAN(1/5) - ARCTAN(1/239)
120 REM ONE DECIMAL DIGIT PER BYTE IN PAGED RAM
130 REM SELECT REGISTER $F280, 4K WINDOW AT $E000-$EFFF
140 REM BLOCK 14 IS THE NATIVE $E000 RAM. MAP IT BACK
150 REM IN (GOSUB 9200) BEFORE EVERY PRINT OR INPUT.
160 R0=62080
170 W0=57344
180 C=-1
190 GOSUB 9200
200 REM BUFFERS: 28 BLOCKS (114688 DIGITS) EACH
210 P0=16
220 T0=44
230 A0=72
240 B0=100
250 DIM P9(6)
260 FOR I=0 TO 6
270 READ P9(I)
280 NEXT I
282 REM ---- MACHINE CODE: PARAMETERS AT $C100, DIVIDE AT $C110
284 P1=HEX("C100")
286 D1=HEX("C110")
288 U0=HEX("BFFD")
289 GOSUB 9500
290 DATA 0,0,4,1,8,4,1
300 INPUT "NUMBER OF DIGITS";N1
310 N=N1+6
320 IF N<=65535 THEN 350
330 PRINT "TOO MANY DIGITS"
340 GOTO 300
350 REM ---- ARCTAN(1/5), STARTING FROM 1/5 = 0.2
360 PRINT "ARCTAN(1/5)"
370 Q3=A0
380 GOSUB 8800
390 Q3=P0
400 GOSUB 8800
410 Q=P0
420 I=0
430 D=2
440 GOSUB 9100
450 X2=25
460 Z=A0
470 GOSUB 7000
480 REM ---- ARCTAN(1/239), 1/239 = 0.0041841 REPEATING
490 GOSUB 9200
500 PRINT "ARCTAN(1/239)"
510 Q3=B0
520 GOSUB 8800
530 Q=P0
540 FOR I=0 TO N-1
550 D=P9(I-7*INT(I/7))
560 GOSUB 9100
570 NEXT I
580 X2=57121
590 Z=B0
600 GOSUB 7000
610 REM ---- PI = 4*(4*ARCTAN(1/5) - ARCTAN(1/239))
620 Q3=A0
630 V=4
640 GOSUB 8700
650 Q2=B0
660 Q3=A0
670 S=-1
680 F=0
690 GOSUB 8300
700 Q3=A0
710 V=4
```

```
720 GOSUB 8700
730 I0=E
740 REM ---- PRINT, 50 DIGITS PER LINE
750 GOSUB 9200
760 PRINT
770 PRINT CHR$(48+I0);".";
780 L$=""
790 FOR I=0 TO N1-1
800 Q=A0
810 GOSUB 9000
820 L$=L$+CHR$(48+D)
830 IF LEN(L$)=50 THEN 860
840 IF I=N1-1 THEN 860
850 GOTO 890
860 GOSUB 9200
862 IF I<50 THEN 870
864 PRINT "  ";
870 PRINT L$
880 L$=""
890 NEXT I
900 GOSUB 9200
910 END
7000 REM ---- ARCTAN SERIES
7010 REM BUFFER P0 HOLDS 1/X, X2=X*X, RESULT ADDED INTO BUFFER Z
7020 REM F = INDEX OF FIRST NON-ZERO DIGIT OF P (SKIPS LEADING ZEROS)
7030 M=0
7040 S=1
7050 F=0
7060 IF F>=N THEN 7290
7070 REM T = P/(2M+1)
7080 Q2=P0
7090 Q3=T0
7100 V=2*M+1
7110 GOSUB 8000
7120 REM Z = Z + S*T
7130 Q2=T0
7140 Q3=Z
7150 GOSUB 8300
7160 REM P = P/X2, IN PLACE
7170 Q2=P0
7180 Q3=P0
7190 V=X2
7200 GOSUB 8000
7210 F=F1
7220 M=M+1
7230 S=-S
7240 IF INT(M/10)*10<>M THEN 7060
7250 GOSUB 9200
7260 PRINT "TERM";M;"  LEADING ZEROS";F
7270 GOTO 7060
7290 RETURN
8000 REM ---- DIVIDE BUFFER Q2 BY V INTO BUFFER Q3, DIGITS F TO N-1
8010 REM F1 RETURNS THE FIRST NON-ZERO QUOTIENT DIGIT (N IF NONE)
8020 REM MACHINE CODE VERSION - SEE PIDIV.ASM
8030 DPOKE P1,Q2
8040 DPOKE P1+2,Q3
8050 DPOKE P1+4,N
8060 DPOKE P1+6,F
8070 DPOKE P1+8,V
8080 DPOKE U0,D1
8090 X=USR(0)
8100 F1=PEEK(P1+12)*256+PEEK(P1+13)
8110 C=14
8120 RETURN
8300 REM ---- ADD S*(BUFFER Q2) INTO BUFFER Q3, DIGITS N-1 DOWN TO F
8310 REM S=1 ADDS, S=-1 SUBTRACTS
8320 E=0
8330 FOR I=N-1 TO F STEP -1
8340 Q=Q2
8350 GOSUB 9000
8360 W1=D
8370 Q=Q3
8380 GOSUB 9000
8390 D=D+S*W1+E
8400 GOSUB 8600
8410 GOSUB 9100
8420 NEXT I
8430 REM CARRY OR BORROW INTO DIGITS ABOVE F
8440 I=F-1
```

```
8450 IF E=0 THEN 8530
8460 IF I<0 THEN 8530
8470 GOSUB 9000
8480 D=D+E
8490 GOSUB 8600
8500 GOSUB 9100
8510 I=I-1
8520 GOTO 8450
8530 RETURN
8600 REM ---- NORMALISE DIGIT D, E RETURNS CARRY (+1) OR BORROW (-1)
8610 E=0
8620 IF D>=0 THEN 8650
8630 D=D+10
8640 E=-1
8650 IF D<10 THEN 8680
8660 D=D-10
8670 E=1
8680 RETURN
8700 REM ---- MULTIPLY BUFFER Q3 BY SMALL INTEGER V, IN PLACE
8710 REM E RETURNS THE CARRY OUT OF THE TOP (INTEGER PART)
8720 E=0
8730 Q=Q3
8740 FOR I=N-1 TO 0 STEP -1
8750 GOSUB 9000
8760 D=D*V+E
8770 E=INT(D/10)
8780 D=D-E*10
8790 GOSUB 9100
8795 NEXT I
8798 RETURN
8800 REM ---- CLEAR BUFFER Q3 TO ZERO
8810 Q=Q3
8820 D=0
8830 FOR I=0 TO N-1
8840 GOSUB 9100
8850 NEXT I
8860 RETURN
9000 REM ---- GET DIGIT I OF BUFFER Q INTO D
9010 K=Q+INT(I/4096)
9020 IF K=C THEN 9050
9030 POKE R0,K
9040 C=K
9050 D=PEEK(W0+I-(K-Q)*4096)
9060 RETURN
9100 REM ---- PUT D INTO DIGIT I OF BUFFER Q
9110 K=Q+INT(I/4096)
9120 IF K=C THEN 9150
9130 POKE R0,K
9140 C=K
9150 POKE W0+I-(K-Q)*4096,D
9160 RETURN
9200 REM ---- MAP NATIVE BLOCK 14 BACK IN BEFORE PRINT/INPUT
9210 POKE R0,14
9220 C=14
9230 RETURN
9500 REM ---- LOAD MACHINE CODE DIVIDE INTO $C110-$C22C
9510 S9=0
9520 FOR I=D1 TO D1+284
9530 READ D
9540 POKE I,D
9550 S9=S9+D
9560 NEXT I
9570 IF S9=37202 THEN 9600
9580 PRINT "CHECKSUM ERROR IN DATA LINES"
9590 STOP
9600 RETURN
9700 DATA 52,118,252,193,4,253,194,47,127,194,53,127,194,54,127,194
9710 DATA 55,127,194,56,252,193,4,179,193,6,16,35,0,239,253,194
9720 DATA 45,252,193,6,68,68,68,68,187,193,1,183,194,49,252,193
9730 DATA 6,132,15,195,224,0,31,1,252,193,6,68,68,68,68,187
9740 DATA 193,3,183,194,50,252,193,6,132,15,195,224,0,31,2,182
9750 DATA 194,49,183,242,128,230,128,247,194,51,140,240,0,38,6,142
9760 DATA 224,0,124,194,49,120,194,56,121,194,55,121,194,54,252,194
9770 DATA 55,253,194,58,182,194,54,183,194,57,120,194,56,121,194,55
9780 DATA 121,194,54,120,194,56,121,194,55,121,194,54,252,194,55,243
9790 DATA 194,58,253,194,55,182,194,54,185,194,57,183,194,54,252,194
9800 DATA 55,251,194,51,137,0,253,194,55,182,194,54,137,0,183,194
9810 DATA 54,127,194,52,252,194,55,179,193,8,253,194,58,182,194,54
9820 DATA 130,0,37,14,183,194,54,252,194,58,253,194,55,124,194,52
```

```
9830 DATA 32,226,125,194,52,39,17,125,194,53,38,12,252,193,4,179
9840 DATA 194,45,253,194,47,124,194,53,182,194,50,183,242,128,246,194
9850 DATA 52,231,160,16,140,240,0,38,7,16,142,224,0,124,194,50
9860 DATA 252,194,45,131,0,1,253,194,45,16,38,255,66,134,14,183
9870 DATA 242,128,252,194,47,253,193,12,253,191,251,53,246
```

PIDIV.ASM

```
* PIDIV - MACHINE CODE DIVIDE FOR THE MACHIN PI PROGRAM
*
* DIVIDES A DIGIT BUFFER BY A 16-BIT INTEGER, ONE DECIMAL
* DIGIT PER BYTE IN PAGED RAM. REPLACES BASIC LINES 8000-8220.
*
*   DST(F..N-1) = SRC(F..N-1) / V
*
* SRC AND DST MAY BE THE SAME BUFFER (IN-PLACE DIVIDE).
* RETURNS F1, THE INDEX OF THE FIRST NON-ZERO QUOTIENT
* DIGIT (N IF THE QUOTIENT IS ALL ZERO), IN RES.
*
* PAGING: 4K WINDOW AT $E000-$EFFF, BLOCK SELECTED BY
* WRITING $F280. $F280 MUST NEVER BE READ - A READ LATCHES
* THE DATA BUS INTO THE PAGE REGISTER. BLOCK 14 (THE NATIVE
* $E000 RAM) IS MAPPED BACK IN BEFORE RETURNING TO BASIC.
*
* CALLED FROM XBASIC WITH X=USR(0), VECTOR AT $BFFD.

PAGREG  EQU     $F280           PAGE SELECT REGISTER (WRITE ONLY)
WINDOW  EQU     $E000           START OF 4K WINDOW
WINEND  EQU     $F000           FIRST ADDRESS PAST THE WINDOW
NATIVE  EQU     14              BLOCK THAT IS NORMALLY AT $E000
USRARG  EQU     $BFFB           USR ARGUMENT/RESULT (MEMEND-4)

        ORG     $C100

* PARAMETER BLOCK - FILLED BY BASIC WITH DPOKE
SRC     RMB     2               $C100 SOURCE BUFFER FIRST BLOCK
DST     RMB     2               $C102 DEST BUFFER FIRST BLOCK
NN      RMB     2               $C104 N, DIGITS IN BUFFER
FF      RMB     2               $C106 F, FIRST DIGIT TO PROCESS
VV      RMB     2               $C108 V, DIVISOR
SS      RMB     2               $C10A S, ADD/SUB (NOT USED HERE)
RES     RMB     2               $C10C RESULT (F1)
        RMB     2               $C10E SPARE

* START OF CODE AT $C110
DIVIDE  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
        LDD     NN
        STD     F1              F1 = N UNTIL A NON-ZERO DIGIT
        CLR     FOUND
        CLR     RH              REMAINDER = 0
        CLR     RM
        CLR     RL
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
        LBLS    DVEXIT          NOTHING TO DO IF N <= F
        STD     CNT
*
* SET UP SOURCE BLOCK/POINTER FOR INDEX F
        LDD     FF
        LSRA                    A = F / 4096
        LSRA
        LSRA
        LSRA
        ADDA    SRC+1
        STA     SBLK
        LDD     FF
        ANDA    #$0F            OFFSET = F AND $0FFF
        ADDD    #WINDOW
        TFR     D,X             X = SOURCE POINTER
*
* SAME FOR DESTINATION
        LDD     FF
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     FF
```

```
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y             Y = DEST POINTER
*
* MAIN LOOP - ONE DIGIT PER PASS
DVLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X+             FETCH DIGIT
        STB     DIG
        CMPX    #WINEND         OFF THE END OF THE WINDOW?
        BNE     DV1
        LDX     #WINDOW         YES - NEXT BLOCK
        INC     SBLK
*
* R = R * 10 + DIGIT (24 BIT)
DV1     ASL     RL              R = R * 2
        ROL     RM
        ROL     RH
        LDD     RM              T = R * 2
        STD     TM
        LDA     RH
        STA     TH
        ASL     RL              R = R * 8
        ROL     RM
        ROL     RH
        ASL     RL
        ROL     RM
        ROL     RH
        LDD     RM              R = R*8 + R*2
        ADDD    TM
        STD     RM
        LDA     RH
        ADCA    TH
        STA     RH
        LDD     RM              R = R + DIGIT
        ADDB    DIG
        ADCA    #0
        STD     RM
        LDA     RH
        ADCA    #0
        STA     RH
*
* Q = R / V BY REPEATED SUBTRACTION (AT MOST 9 TIMES)
        CLR     QQ
DV2     LDD     RM
        SUBD    VV
        STD     TM
        LDA     RH
        SBCA    #0
        BCS     DV3             BORROW - R < V, Q IS DONE
        STA     RH
        LDD     TM
        STD     RM
        INC     QQ
        BRA     DV2
*
* RECORD THE FIRST NON-ZERO QUOTIENT DIGIT
DV3     TST     QQ
        BEQ     DV4
        TST     FOUND
        BNE     DV4
        LDD     NN              INDEX = N - CNT
        SUBD    CNT
        STD     F1
        INC     FOUND
*
* STORE THE QUOTIENT DIGIT
DV4     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     QQ
        STB     ,Y+
        CMPY    #WINEND
        BNE     DV5
        LDY     #WINDOW
        INC     DBLK
*
DV5     LDD     CNT             COUNT DOWN
        SUBD    #1
        STD     CNT
```

```
        LBNE    DVLOOP
*
DVEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK FOR BASIC
        STA     PAGREG
        LDD     F1
        STD     RES
        STD     USRARG          ALSO RETURN IT AS THE USR VALUE
        PULS    D,X,Y,U,PC      RESTORE REGISTERS AND RETURN

* WORKING STORAGE
CNT     RMB     2               DIGITS LEFT TO DO
F1      RMB     2               FIRST NON-ZERO QUOTIENT INDEX
SBLK    RMB     1               CURRENT SOURCE BLOCK
DBLK    RMB     1               CURRENT DEST BLOCK
DIG     RMB     1               DIGIT JUST FETCHED
QQ      RMB     1               QUOTIENT DIGIT
FOUND   RMB     1               NON-ZERO DIGIT SEEN FLAG
RH      RMB     1               24-BIT REMAINDER
RM      RMB     1
RL      RMB     1
TH      RMB     1               24-BIT TEMPORARY
TM      RMB     1
TL      RMB     1

        END
```

Appendix C — Version 3: machine-code divide and add/subtract

Files: PI_MACHIN.BAS (version 3) and PIMATH.ASM (version 3)

Performance and layout: 100 digits: 25 s (measured), 29× version 1. 1,000 digits: 7 min 45 s (measured). Divide at $C110, ADDSUB at $C240; loader covers $C110–$C33C (557 bytes, checksum 67797).

PI_MACHIN.BAS — version 3

```
100 REM PI BY MACHIN'S FORMULA - PURE BASIC VERSION
110 REM PI/4 = 4*ARCTAN(1/5) - ARCTAN(1/239)
120 REM ONE DECIMAL DIGIT PER BYTE IN PAGED RAM
130 REM SELECT REGISTER $F280, 4K WINDOW AT $E000-$EFFF
140 REM BLOCK 14 IS THE NATIVE $E000 RAM. MAP IT BACK
150 REM IN (GOSUB 9200) BEFORE EVERY PRINT OR INPUT.
160 R0=62080
170 W0=57344
180 C=-1
190 GOSUB 9200
200 REM BUFFERS: 28 BLOCKS (114688 DIGITS) EACH
210 P0=16
220 T0=44
230 A0=72
240 B0=100
250 DIM P9(6)
260 FOR I=0 TO 6
270 READ P9(I)
280 NEXT I
282 REM ---- MACHINE CODE: PARAMETERS $C100, DIVIDE $C110, ADD/SUB $C240
284 P1=HEX("C100")
286 D1=HEX("C110")
287 A1=HEX("C240")
288 U0=HEX("BFFD")
289 GOSUB 9500
290 DATA 0,0,4,1,8,4,1
300 INPUT "NUMBER OF DIGITS";N1
310 N=N1+6
320 IF N<=65535 THEN 350
330 PRINT "TOO MANY DIGITS"
340 GOTO 300
350 REM ---- ARCTAN(1/5), STARTING FROM 1/5 = 0.2
360 PRINT "ARCTAN(1/5)"
370 Q3=A0
380 GOSUB 8800
390 Q3=P0
400 GOSUB 8800
410 Q=P0
420 I=0
430 D=2
440 GOSUB 9100
450 X2=25
460 Z=A0
470 GOSUB 7000
480 REM ---- ARCTAN(1/239), 1/239 = 0.0041841 REPEATING
490 GOSUB 9200
500 PRINT "ARCTAN(1/239)"
510 Q3=B0
520 GOSUB 8800
530 Q=P0
540 FOR I=0 TO N-1
550 D=P9(I-7*INT(I/7))
560 GOSUB 9100
570 NEXT I
580 X2=57121
590 Z=B0
600 GOSUB 7000
610 REM ---- PI = 4*(4*ARCTAN(1/5) - ARCTAN(1/239))
620 Q3=A0
630 V=4
640 GOSUB 8700
650 Q2=B0
660 Q3=A0
```

```
670 S=-1
680 F=0
690 GOSUB 8300
700 Q3=A0
710 V=4
720 GOSUB 8700
730 I0=E
740 REM ---- PRINT, 50 DIGITS PER LINE
750 GOSUB 9200
760 PRINT
770 PRINT CHR$(48+I0);".";
780 L$=""
790 FOR I=0 TO N1-1
800 Q=A0
810 GOSUB 9000
820 L$=L$+CHR$(48+D)
830 IF LEN(L$)=50 THEN 860
840 IF I=N1-1 THEN 860
850 GOTO 890
860 GOSUB 9200
862 IF I<50 THEN 870
864 PRINT "  ";
870 PRINT L$
880 L$=""
890 NEXT I
900 GOSUB 9200
910 END
7000 REM ---- ARCTAN SERIES
7010 REM BUFFER P0 HOLDS 1/X, X2=X*X, RESULT ADDED INTO BUFFER Z
7020 REM F = INDEX OF FIRST NON-ZERO DIGIT OF P (SKIPS LEADING ZEROS)
7030 M=0
7040 S=1
7050 F=0
7060 IF F>=N THEN 7290
7070 REM T = P/(2M+1)
7080 Q2=P0
7090 Q3=T0
7100 V=2*M+1
7110 GOSUB 8000
7120 REM Z = Z + S*T
7130 Q2=T0
7140 Q3=Z
7150 GOSUB 8300
7160 REM P = P/X2, IN PLACE
7170 Q2=P0
7180 Q3=P0
7190 V=X2
7200 GOSUB 8000
7210 F=F1
7220 M=M+1
7230 S=-S
7240 IF INT(M/10)*10<>M THEN 7060
7250 GOSUB 9200
7260 PRINT "TERM";M;"  LEADING ZEROS";F
7270 GOTO 7060
7290 RETURN
8000 REM ---- DIVIDE BUFFER Q2 BY V INTO BUFFER Q3, DIGITS F TO N-1
8010 REM F1 RETURNS THE FIRST NON-ZERO QUOTIENT DIGIT (N IF NONE)
8020 REM MACHINE CODE VERSION - SEE PIMATH.ASM
8030 DPOKE P1,Q2
8040 DPOKE P1+2,Q3
8050 DPOKE P1+4,N
8060 DPOKE P1+6,F
8070 DPOKE P1+8,V
8080 DPOKE U0,D1
8090 X=USR(0)
8100 F1=PEEK(P1+12)*256+PEEK(P1+13)
8110 C=14
8120 RETURN
8300 REM ---- ADD S*(BUFFER Q2) INTO BUFFER Q3, DIGITS N-1 DOWN TO F
8310 REM S=1 ADDS, S=-1 SUBTRACTS. MACHINE CODE - SEE PIMATH.ASM
8320 DPOKE P1,Q2
8330 DPOKE P1+2,Q3
8340 DPOKE P1+4,N
8350 DPOKE P1+6,F
8360 DPOKE P1+10,(1-S)/2
8370 DPOKE U0,A1
8380 X=USR(0)
8390 C=14
```

```
8400 RETURN
8600 REM ---- NORMALISE DIGIT D, E RETURNS CARRY (+1) OR BORROW (-1)
8610 E=0
8620 IF D>=0 THEN 8650
8630 D=D+10
8640 E=-1
8650 IF D<10 THEN 8680
8660 D=D-10
8670 E=1
8680 RETURN
8700 REM ---- MULTIPLY BUFFER Q3 BY SMALL INTEGER V, IN PLACE
8710 REM E RETURNS THE CARRY OUT OF THE TOP (INTEGER PART)
8720 E=0
8730 Q=Q3
8740 FOR I=N-1 TO 0 STEP -1
8750 GOSUB 9000
8760 D=D*V+E
8770 E=INT(D/10)
8780 D=D-E*10
8790 GOSUB 9100
8795 NEXT I
8798 RETURN
8800 REM ---- CLEAR BUFFER Q3 TO ZERO
8810 Q=Q3
8820 D=0
8830 FOR I=0 TO N-1
8840 GOSUB 9100
8850 NEXT I
8860 RETURN
9000 REM ---- GET DIGIT I OF BUFFER Q INTO D
9010 K=Q+INT(I/4096)
9020 IF K=C THEN 9050
9030 POKE R0,K
9040 C=K
9050 D=PEEK(W0+I-(K-Q)*4096)
9060 RETURN
9100 REM ---- PUT D INTO DIGIT I OF BUFFER Q
9110 K=Q+INT(I/4096)
9120 IF K=C THEN 9150
9130 POKE R0,K
9140 C=K
9150 POKE W0+I-(K-Q)*4096,D
9160 RETURN
9200 REM ---- MAP NATIVE BLOCK 14 BACK IN BEFORE PRINT/INPUT
9210 POKE R0,14
9220 C=14
9230 RETURN
9500 REM ---- LOAD MACHINE CODE INTO $C110-$C33C
9510 S9=0
9520 FOR I=D1 TO D1+556
9530 READ D
9540 POKE I,D
9550 S9=S9+D
9560 NEXT I
9570 IF S9=67797 THEN 9600
9580 PRINT "CHECKSUM ERROR IN DATA LINES"
9590 STOP
9600 RETURN
9700 DATA 52,118,252,193,4,253,194,47,127,194,53,127,194,54,127,194
9710 DATA 55,127,194,56,252,193,4,179,193,6,16,35,0,239,253,194
9720 DATA 45,252,193,6,68,68,68,68,187,193,1,183,194,49,252,193
9730 DATA 6,132,15,195,224,0,31,1,252,193,6,68,68,68,68,187
9740 DATA 193,3,183,194,50,252,193,6,132,15,195,224,0,31,2,182
9750 DATA 194,49,183,242,128,230,128,247,194,51,140,240,0,38,6,142
9760 DATA 224,0,124,194,49,120,194,56,121,194,55,121,194,54,252,194
9770 DATA 55,253,194,58,182,194,54,183,194,57,120,194,56,121,194,55
9780 DATA 121,194,54,120,194,56,121,194,55,121,194,54,252,194,55,243
9790 DATA 194,58,253,194,55,182,194,54,185,194,57,183,194,54,252,194
9800 DATA 55,251,194,51,137,0,253,194,55,182,194,54,137,0,183,194
9810 DATA 54,127,194,52,252,194,55,179,193,8,253,194,58,182,194,54
9820 DATA 130,0,37,14,183,194,54,252,194,58,253,194,55,124,194,52
9830 DATA 32,226,125,194,52,39,17,125,194,53,38,12,252,193,4,179
9840 DATA 194,45,253,194,47,124,194,53,182,194,50,183,242,128,246,194
9850 DATA 52,231,160,16,140,240,0,38,7,16,142,224,0,124,194,50
9860 DATA 252,194,45,131,0,1,253,194,45,16,38,255,66,134,14,183
9870 DATA 242,128,252,194,47,253,193,12,253,191,251,53,246,0,0,0
9880 DATA 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
9890 DATA 52,118,127,194,60,182,193,11,183,194,61,252,193,4,179,193
9900 DATA 6,16,35,0,225,253,194,45,252,193,4,131,0,1,253,194
```

```
9910 DATA 62,68,68,68,68,187,193,1,183,194,49,252,194,62,132,15
9920 DATA 195,224,0,31,1,252,194,62,68,68,68,68,187,193,3,183
9930 DATA 194,50,252,194,62,132,15,195,224,0,31,2,182,194,49,183
9940 DATA 242,128,230,132,247,194,51,48,31,140,223,255,38,6,142,239
9950 DATA 255,122,194,49,182,194,50,183,242,128,230,164,125,194,61,38
9960 DATA 16,251,194,51,251,194,60,193,10,37,20,192,10,134,1,32
9970 DATA 15,240,194,51,240,194,60,44,6,203,10,134,1,32,1,79
9980 DATA 183,194,60,231,164,49,63,16,140,223,255,38,7,16,142,239
9990 DATA 255,122,194,50,252,194,45,131,0,1,253,194,45,38,157,125
10000 DATA 194,60,39,66,252,193,6,39,61,253,194,45,182,194,50,183
10010 DATA 242,128,230,164,125,194,61,38,8,92,193,10,37,38,95,32
10020 DATA 5,90,42,32,198,9,231,164,49,63,16,140,223,255,38,7
10030 DATA 16,142,239,255,122,194,50,252,194,45,131,0,1,253,194,45
10040 DATA 38,202,32,2,231,164,134,14,183,242,128,53,246
```

PIMATH.ASM — version 3

```
* PIMATH - MACHINE CODE ROUTINES FOR THE MACHIN PI PROGRAM
*
* $C110 DIVIDE  - REPLACES BASIC LINES 8000-8220
* $C240 ADDSUB  - REPLACES BASIC LINES 8300-8530
*
*****************************************************
* DIVIDE
*
* DIVIDES A DIGIT BUFFER BY A 16-BIT INTEGER, ONE DECIMAL
* DIGIT PER BYTE IN PAGED RAM. REPLACES BASIC LINES 8000-8220.
*
*   DST(F..N-1) = SRC(F..N-1) / V
*
* SRC AND DST MAY BE THE SAME BUFFER (IN-PLACE DIVIDE).
* RETURNS F1, THE INDEX OF THE FIRST NON-ZERO QUOTIENT
* DIGIT (N IF THE QUOTIENT IS ALL ZERO), IN RES.
*
* PAGING: 4K WINDOW AT $E000-$EFFF, BLOCK SELECTED BY
* WRITING $F280. $F280 MUST NEVER BE READ - A READ LATCHES
* THE DATA BUS INTO THE PAGE REGISTER. BLOCK 14 (THE NATIVE
* $E000 RAM) IS MAPPED BACK IN BEFORE RETURNING TO BASIC.
*
* CALLED FROM XBASIC WITH X=USR(0), VECTOR AT $BFFD.

PAGREG  EQU     $F280           PAGE SELECT REGISTER (WRITE ONLY)
WINDOW  EQU     $E000           START OF 4K WINDOW
WINEND  EQU     $F000           FIRST ADDRESS PAST THE WINDOW
NATIVE  EQU     14              BLOCK THAT IS NORMALLY AT $E000
USRARG  EQU     $BFFB           USR ARGUMENT/RESULT (MEMEND-4)

        ORG     $C100

* PARAMETER BLOCK - FILLED BY BASIC WITH DPOKE
SRC     RMB     2               $C100 SOURCE BUFFER FIRST BLOCK
DST     RMB     2               $C102 DEST BUFFER FIRST BLOCK
NN      RMB     2               $C104 N, DIGITS IN BUFFER
FF      RMB     2               $C106 F, FIRST DIGIT TO PROCESS
VV      RMB     2               $C108 V, DIVISOR
SS      RMB     2               $C10A 0 = ADD, 1 = SUBTRACT
RES     RMB     2               $C10C RESULT (F1)
        RMB     2               $C10E SPARE

* START OF CODE AT $C110
DIVIDE  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
        LDD     NN
        STD     F1              F1 = N UNTIL A NON-ZERO DIGIT
        CLR     FOUND
        CLR     RH              REMAINDER = 0
        CLR     RM
        CLR     RL
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
        LBLS    DVEXIT          NOTHING TO DO IF N <= F
        STD     CNT
*
* SET UP SOURCE BLOCK/POINTER FOR INDEX F
        LDD     FF
        LSRA                    A = F / 4096
        LSRA
        LSRA
        LSRA
        ADDA    SRC+1
```

```
        STA     SBLK
        LDD     FF
        ANDA    #$0F            OFFSET = F AND $0FFF
        ADDD    #WINDOW
        TFR     D,X             X = SOURCE POINTER
*
* SAME FOR DESTINATION
        LDD     FF
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     FF
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y             Y = DEST POINTER
*
* MAIN LOOP - ONE DIGIT PER PASS
DVLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X+             FETCH DIGIT
        STB     DIG
        CMPX    #WINEND         OFF THE END OF THE WINDOW?
        BNE     DV1
        LDX     #WINDOW         YES - NEXT BLOCK
        INC     SBLK
*
* R = R * 10 + DIGIT (24 BIT)
DV1     ASL     RL              R = R * 2
        ROL     RM
        ROL     RH
        LDD     RM              T = R * 2
        STD     TM
        LDA     RH
        STA     TH
        ASL     RL              R = R * 8
        ROL     RM
        ROL     RH
        ASL     RL
        ROL     RM
        ROL     RH
        LDD     RM              R = R*8 + R*2
        ADDD    TM
        STD     RM
        LDA     RH
        ADCA    TH
        STA     RH
        LDD     RM              R = R + DIGIT
        ADDB    DIG
        ADCA    #0
        STD     RM
        LDA     RH
        ADCA    #0
        STA     RH
*
* Q = R / V BY REPEATED SUBTRACTION (AT MOST 9 TIMES)
        CLR     QQ
DV2     LDD     RM
        SUBD    VV
        STD     TM
        LDA     RH
        SBCA    #0
        BCS     DV3             BORROW - R < V, Q IS DONE
        STA     RH
        LDD     TM
        STD     RM
        INC     QQ
        BRA     DV2
*
* RECORD THE FIRST NON-ZERO QUOTIENT DIGIT
DV3     TST     QQ
        BEQ     DV4
        TST     FOUND
        BNE     DV4
        LDD     NN              INDEX = N - CNT
        SUBD    CNT
        STD     F1
```

```
        INC     FOUND
*
* STORE THE QUOTIENT DIGIT
DV4     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     QQ
        STB     ,Y+
        CMPY    #WINEND
        BNE     DV5
        LDY     #WINDOW
        INC     DBLK
*
DV5     LDD     CNT             COUNT DOWN
        SUBD    #1
        STD     CNT
        LBNE    DVLOOP
*
DVEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK FOR BASIC
        STA     PAGREG
        LDD     F1
        STD     RES
        STD     USRARG          ALSO RETURN IT AS THE USR VALUE
        PULS    D,X,Y,U,PC      RESTORE REGISTERS AND RETURN

* WORKING STORAGE
CNT     RMB     2               DIGITS LEFT TO DO
F1      RMB     2               FIRST NON-ZERO QUOTIENT INDEX
SBLK    RMB     1               CURRENT SOURCE BLOCK
DBLK    RMB     1               CURRENT DEST BLOCK
DIG     RMB     1               DIGIT JUST FETCHED
QQ      RMB     1               QUOTIENT DIGIT
FOUND   RMB     1               NON-ZERO DIGIT SEEN FLAG
RH      RMB     1               24-BIT REMAINDER
RM      RMB     1
RL      RMB     1
TH      RMB     1               24-BIT TEMPORARY
TM      RMB     1
TL      RMB     1
CY      RMB     1               CARRY OR BORROW (0/1)
SUBF    RMB     1               SUBTRACT FLAG
IDX     RMB     2               WORKING DIGIT INDEX

*******************************************************
* ADDSUB - ADD OR SUBTRACT A DIGIT BUFFER INTO ANOTHER
*
*   DST(I) = DST(I) + SRC(I)   FOR I = N-1 DOWN TO F  (SS=0)
*   DST(I) = DST(I) - SRC(I)                          (SS=1)
*
* THEN ANY CARRY OR BORROW IS PROPAGATED INTO DST BELOW
* INDEX F, AS FAR AS IT GOES (TOWARDS INDEX 0).
* SRC MUST NOT BE THE SAME BUFFER AS DST.

        ORG     $C240

ADDSUB  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
        CLR     CY
        LDA     SS+1
        STA     SUBF
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
        LBLS    ASEXIT          NOTHING TO DO IF N <= F
        STD     CNT
        LDD     NN              START AT INDEX N-1
        SUBD    #1
        STD     IDX
*
* SOURCE BLOCK/POINTER FOR INDEX N-1
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    SRC+1
        STA     SBLK
        LDD     IDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,X
*
* DEST BLOCK/POINTER FOR INDEX N-1
```

```
        LDD     IDX
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     IDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y
*
* MAIN LOOP - ONE DIGIT PER PASS, WORKING DOWNWARDS
ASLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X
        STB     DIG
        LEAX    -1,X            STEP BACK
        CMPX    #WINDOW-1       OFF THE START OF THE WINDOW?
        BNE     AS1
        LDX     #WINEND-1       YES - PREVIOUS BLOCK
        DEC     SBLK
*
AS1     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     ,Y              B = DEST DIGIT
        TST     SUBF
        BNE     ASSUB
*
        ADDB    DIG             ADD: B = DEST + SRC + CARRY
        ADDB    CY
        CMPB    #10
        BLO     ASNC
        SUBB    #10
        LDA     #1              CARRY
        BRA     AS2
*
ASSUB   SUBB    DIG             SUBTRACT: B = DEST - SRC - BORROW
        SUBB    CY
        BGE     ASNC
        ADDB    #10
        LDA     #1              BORROW
        BRA     AS2
*
ASNC    CLRA                    NO CARRY OR BORROW
AS2     STA     CY
        STB     ,Y              STORE RESULT DIGIT
        LEAY    -1,Y
        CMPY    #WINDOW-1
        BNE     AS3
        LDY     #WINEND-1
        DEC     DBLK
*
AS3     LDD     CNT
        SUBD    #1
        STD     CNT
        BNE     ASLOOP
*
* PROPAGATE CARRY/BORROW INTO DIGITS F-1 DOWN TO 0
* Y AND DBLK ALREADY POINT AT INDEX F-1
        TST     CY
        BEQ     ASEXIT
        LDD     FF
        BEQ     ASEXIT          NOTHING ABOVE INDEX 0
        STD     CNT
*
PRLOOP  LDA     DBLK
        STA     PAGREG
        LDB     ,Y
        TST     SUBF
        BNE     PRSUB
        INCB                    ADD CARRY
        CMPB    #10
        BLO     PRLAST
        CLRB                    9+1 = 0, CARRY ON
        BRA     PRNEXT
PRSUB   DECB                    SUBTRACT BORROW
        BPL     PRLAST
        LDB     #9              0-1 = 9, BORROW ON
```

```
PRNEXT  STB     ,Y
        LEAY    -1,Y
        CMPY    #WINDOW-1
        BNE     PR1
        LDY     #WINEND-1
        DEC     DBLK
PR1     LDD     CNT
        SUBD    #1
        STD     CNT
        BNE     PRLOOP
        BRA     ASEXIT          RAN OFF THE TOP - IGNORE
*
PRLAST  STB     ,Y              CARRY/BORROW ABSORBED
*
ASEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK FOR BASIC
        STA     PAGREG
        PULS    D,X,Y,U,PC

        END
```

Appendix D — Version 4: fast divide path

Files: PI_MACHIN.BAS (version 4) and PIMATH.ASM (version 4)

Performance and layout: 1,000 digits: 6 min 00 s (measured), 174× version 1. Divide at $C110 with fast and general paths, ADDSUB at $C300; loader covers $C110–$C3FC (749 bytes, checksum 102058).

PI_MACHIN.BAS — version 4

```
100 REM PI BY MACHIN'S FORMULA - PURE BASIC VERSION
110 REM PI/4 = 4*ARCTAN(1/5) - ARCTAN(1/239)
120 REM ONE DECIMAL DIGIT PER BYTE IN PAGED RAM
130 REM SELECT REGISTER $F280, 4K WINDOW AT $E000-$EFFF
140 REM BLOCK 14 IS THE NATIVE $E000 RAM. MAP IT BACK
150 REM IN (GOSUB 9200) BEFORE EVERY PRINT OR INPUT.
160 R0=62080
170 W0=57344
180 C=-1
190 GOSUB 9200
200 REM BUFFERS: 28 BLOCKS (114688 DIGITS) EACH
210 P0=16
220 T0=44
230 A0=72
240 B0=100
250 DIM P9(6)
260 FOR I=0 TO 6
270 READ P9(I)
280 NEXT I
282 REM ---- MACHINE CODE: PARAMETERS $C100, DIVIDE $C110, ADD/SUB $C300
284 P1=HEX("C100")
286 D1=HEX("C110")
287 A1=HEX("C300")
288 U0=HEX("BFFD")
289 GOSUB 9500
290 DATA 0,0,4,1,8,4,1
300 INPUT "NUMBER OF DIGITS";N1
310 N=N1+6
320 IF N<=65535 THEN 350
330 PRINT "TOO MANY DIGITS"
340 GOTO 300
350 REM ---- ARCTAN(1/5), STARTING FROM 1/5 = 0.2
360 PRINT "ARCTAN(1/5)"
370 Q3=A0
380 GOSUB 8800
390 Q3=P0
400 GOSUB 8800
410 Q=P0
420 I=0
430 D=2
440 GOSUB 9100
450 X2=25
460 Z=A0
470 GOSUB 7000
480 REM ---- ARCTAN(1/239), 1/239 = 0.0041841 REPEATING
490 GOSUB 9200
500 PRINT "ARCTAN(1/239)"
510 Q3=B0
520 GOSUB 8800
530 Q=P0
540 FOR I=0 TO N-1
550 D=P9(I-7*INT(I/7))
560 GOSUB 9100
570 NEXT I
580 X2=57121
590 Z=B0
600 GOSUB 7000
610 REM ---- PI = 4*(4*ARCTAN(1/5) - ARCTAN(1/239))
620 Q3=A0
630 V=4
640 GOSUB 8700
650 Q2=B0
660 Q3=A0
670 S=-1
680 F=0
```

```
690 GOSUB 8300
700 Q3=A0
710 V=4
720 GOSUB 8700
730 I0=E
740 REM ---- PRINT, 50 DIGITS PER LINE
750 GOSUB 9200
760 PRINT
770 PRINT CHR$(48+I0);".";
780 L$=""
790 FOR I=0 TO N1-1
800 Q=A0
810 GOSUB 9000
820 L$=L$+CHR$(48+D)
830 IF LEN(L$)=50 THEN 860
840 IF I=N1-1 THEN 860
850 GOTO 890
860 GOSUB 9200
862 IF I<50 THEN 870
864 PRINT "  ";
870 PRINT L$
880 L$=""
890 NEXT I
900 GOSUB 9200
910 END
7000 REM ---- ARCTAN SERIES
7010 REM BUFFER P0 HOLDS 1/X, X2=X*X, RESULT ADDED INTO BUFFER Z
7020 REM F = INDEX OF FIRST NON-ZERO DIGIT OF P (SKIPS LEADING ZEROS)
7030 M=0
7040 S=1
7050 F=0
7060 IF F>=N THEN 7290
7070 REM T = P/(2M+1)
7080 Q2=P0
7090 Q3=T0
7100 V=2*M+1
7110 GOSUB 8000
7120 REM Z = Z + S*T
7130 Q2=T0
7140 Q3=Z
7150 GOSUB 8300
7160 REM P = P/X2, IN PLACE
7170 Q2=P0
7180 Q3=P0
7190 V=X2
7200 GOSUB 8000
7210 F=F1
7220 M=M+1
7230 S=-S
7240 IF INT(M/10)*10<>M THEN 7060
7250 GOSUB 9200
7260 PRINT "TERM";M;"  LEADING ZEROS";F
7270 GOTO 7060
7290 RETURN
8000 REM ---- DIVIDE BUFFER Q2 BY V INTO BUFFER Q3, DIGITS F TO N-1
8010 REM F1 RETURNS THE FIRST NON-ZERO QUOTIENT DIGIT (N IF NONE)
8020 REM MACHINE CODE VERSION - SEE PIMATH.ASM
8030 DPOKE P1,Q2
8040 DPOKE P1+2,Q3
8050 DPOKE P1+4,N
8060 DPOKE P1+6,F
8070 DPOKE P1+8,V
8080 DPOKE U0,D1
8090 X=USR(0)
8100 F1=PEEK(P1+12)*256+PEEK(P1+13)
8110 C=14
8120 RETURN
8300 REM ---- ADD S*(BUFFER Q2) INTO BUFFER Q3, DIGITS N-1 DOWN TO F
8310 REM S=1 ADDS, S=-1 SUBTRACTS. MACHINE CODE - SEE PIMATH.ASM
8320 DPOKE P1,Q2
8330 DPOKE P1+2,Q3
8340 DPOKE P1+4,N
8350 DPOKE P1+6,F
8360 DPOKE P1+10,(1-S)/2
8370 DPOKE U0,A1
8380 X=USR(0)
8390 C=14
8400 RETURN
8600 REM ---- NORMALISE DIGIT D, E RETURNS CARRY (+1) OR BORROW (-1)
```

```
8610 E=0
8620 IF D>=0 THEN 8650
8630 D=D+10
8640 E=-1
8650 IF D<10 THEN 8680
8660 D=D-10
8670 E=1
8680 RETURN
8700 REM ---- MULTIPLY BUFFER Q3 BY SMALL INTEGER V, IN PLACE
8710 REM E RETURNS THE CARRY OUT OF THE TOP (INTEGER PART)
8720 E=0
8730 Q=Q3
8740 FOR I=N-1 TO 0 STEP -1
8750 GOSUB 9000
8760 D=D*V+E
8770 E=INT(D/10)
8780 D=D-E*10
8790 GOSUB 9100
8795 NEXT I
8798 RETURN
8800 REM ---- CLEAR BUFFER Q3 TO ZERO
8810 Q=Q3
8820 D=0
8830 FOR I=0 TO N-1
8840 GOSUB 9100
8850 NEXT I
8860 RETURN
9000 REM ---- GET DIGIT I OF BUFFER Q INTO D
9010 K=Q+INT(I/4096)
9020 IF K=C THEN 9050
9030 POKE R0,K
9040 C=K
9050 D=PEEK(W0+I-(K-Q)*4096)
9060 RETURN
9100 REM ---- PUT D INTO DIGIT I OF BUFFER Q
9110 K=Q+INT(I/4096)
9120 IF K=C THEN 9150
9130 POKE R0,K
9140 C=K
9150 POKE W0+I-(K-Q)*4096,D
9160 RETURN
9200 REM ---- MAP NATIVE BLOCK 14 BACK IN BEFORE PRINT/INPUT
9210 POKE R0,14
9220 C=14
9230 RETURN
9500 REM ---- LOAD MACHINE CODE INTO $C110-$C3FC
9510 S9=0
9520 FOR I=D1 TO D1+748
9530 READ D
9540 POKE I,D
9550 S9=S9+D
9560 NEXT I
9570 IF S9=102058 THEN 9600
9580 PRINT "CHECKSUM ERROR IN DATA LINES"
9590 STOP
9600 RETURN
9700 DATA 52,118,252,193,4,253,194,206,127,194,212,127,194,213,127,194
9710 DATA 214,127,194,215,252,193,4,179,193,6,16,35,1,5,253,194
9720 DATA 204,252,193,6,68,68,68,68,187,193,1,183,194,208,252,193
9730 DATA 6,132,15,195,224,0,31,1,252,193,6,68,68,68,68,187
9740 DATA 193,3,183,194,209,252,193,6,132,15,195,224,0,31,2,252
9750 DATA 193,8,16,131,25,153,34,13,88,73,88,73,243,193,8,253
9760 DATA 194,223,22,0,206,182,194,208,183,242,128,230,128,247,194,210
9770 DATA 140,240,0,38,6,142,224,0,124,194,208,120,194,215,121,194
9780 DATA 214,121,194,213,252,194,214,253,194,217,182,194,213,183,194,216
9790 DATA 120,194,215,121,194,214,121,194,213,120,194,215,121,194,214,121
9800 DATA 194,213,252,194,214,243,194,217,253,194,214,182,194,213,185,194
9810 DATA 216,183,194,213,252,194,214,251,194,210,137,0,253,194,214,182
9820 DATA 194,213,137,0,183,194,213,127,194,211,252,194,214,179,193,8
9830 DATA 253,194,217,182,194,213,130,0,37,14,183,194,213,252,194,217
9840 DATA 253,194,214,124,194,211,32,226,125,194,211,39,17,125,194,212
9850 DATA 38,12,252,193,4,179,194,204,253,194,206,124,194,212,182,194
9860 DATA 209,183,242,128,246,194,211,231,160,16,140,240,0,38,7,16
9870 DATA 142,224,0,124,194,209,252,194,204,131,0,1,253,194,204,16
9880 DATA 38,255,66,134,14,183,242,128,252,194,206,253,193,12,253,191
9890 DATA 251,53,246,204,0,0,88,73,253,194,225,88,73,88,73,243
9900 DATA 194,225,253,194,225,182,194,208,183,242,128,230,128,79,243,194
9910 DATA 225,140,240,0,38,6,142,224,0,124,194,208,179,194,223,36
9920 DATA 8,243,194,223,206,0,0,32,3,206,0,5,179,193,8,37
```

```
9930 DATA 4,51,65,32,247,243,193,8,253,194,227,125,194,212,38,18
9940 DATA 17,131,0,0,39,12,252,193,4,179,194,204,253,194,206,124
9950 DATA 194,212,182,194,209,183,242,128,31,48,231,160,16,140,240,0
9960 DATA 38,7,16,142,224,0,124,194,209,252,194,204,131,0,1,253
9970 DATA 194,204,16,39,255,109,252,194,227,22,255,122,0,0,0,0
9980 DATA 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
9990 DATA 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
10000 DATA 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
10010 DATA 52,118,127,194,219,182,193,11,183,194,220,252,193,4,179,193
10020 DATA 6,16,35,0,225,253,194,204,252,193,4,131,0,1,253,194
10030 DATA 221,68,68,68,68,187,193,1,183,194,208,252,194,221,132,15
10040 DATA 195,224,0,31,1,252,194,221,68,68,68,68,187,193,3,183
10050 DATA 194,209,252,194,221,132,15,195,224,0,31,2,182,194,208,183
10060 DATA 242,128,230,132,247,194,210,48,31,140,223,255,38,6,142,239
10070 DATA 255,122,194,208,182,194,209,183,242,128,230,164,125,194,220,38
10080 DATA 16,251,194,210,251,194,219,193,10,37,20,192,10,134,1,32
10090 DATA 15,240,194,210,240,194,219,44,6,203,10,134,1,32,1,79
10100 DATA 183,194,219,231,164,49,63,16,140,223,255,38,7,16,142,239
10110 DATA 255,122,194,209,252,194,204,131,0,1,253,194,204,38,157,125
10120 DATA 194,219,39,66,252,193,6,39,61,253,194,204,182,194,209,183
10130 DATA 242,128,230,164,125,194,220,38,8,92,193,10,37,38,95,32
10140 DATA 5,90,42,32,198,9,231,164,49,63,16,140,223,255,38,7
10150 DATA 16,142,239,255,122,194,209,252,194,204,131,0,1,253,194,204
10160 DATA 38,202,32,2,231,164,134,14,183,242,128,53,246
```

PIMATH.ASM — version 4

```
* PIMATH - MACHINE CODE ROUTINES FOR THE MACHIN PI PROGRAM
*
* $C110 DIVIDE  - REPLACES BASIC LINES 8000-8220
* $C300 ADDSUB  - REPLACES BASIC LINES 8300-8530
*
******************************************************
* DIVIDE
*
* DIVIDES A DIGIT BUFFER BY A 16-BIT INTEGER, ONE DECIMAL
* DIGIT PER BYTE IN PAGED RAM. REPLACES BASIC LINES 8000-8220.
*
*   DST(F..N-1) = SRC(F..N-1) / V
*
* SRC AND DST MAY BE THE SAME BUFFER (IN-PLACE DIVIDE).
* RETURNS F1, THE INDEX OF THE FIRST NON-ZERO QUOTIENT
* DIGIT (N IF THE QUOTIENT IS ALL ZERO), IN RES.
*
* PAGING: 4K WINDOW AT $E000-$EFFF, BLOCK SELECTED BY
* WRITING $F280. $F280 MUST NEVER BE READ - A READ LATCHES
* THE DATA BUS INTO THE PAGE REGISTER. BLOCK 14 (THE NATIVE
* $E000 RAM) IS MAPPED BACK IN BEFORE RETURNING TO BASIC.
*
* CALLED FROM XBASIC WITH X=USR(0), VECTOR AT $BFFD.

PAGREG  EQU     $F280           PAGE SELECT REGISTER (WRITE ONLY)
WINDOW  EQU     $E000           START OF 4K WINDOW
WINEND  EQU     $F000           FIRST ADDRESS PAST THE WINDOW
NATIVE  EQU     14              BLOCK THAT IS NORMALLY AT $E000
USRARG  EQU     $BFFB           USR ARGUMENT/RESULT (MEMEND-4)

        ORG     $C100

* PARAMETER BLOCK - FILLED BY BASIC WITH DPOKE
SRC     RMB     2               $C100 SOURCE BUFFER FIRST BLOCK
DST     RMB     2               $C102 DEST BUFFER FIRST BLOCK
NN      RMB     2               $C104 N, DIGITS IN BUFFER
FF      RMB     2               $C106 F, FIRST DIGIT TO PROCESS
VV      RMB     2               $C108 V, DIVISOR
SS      RMB     2               $C10A 0 = ADD, 1 = SUBTRACT
RES     RMB     2               $C10C RESULT (F1)
        RMB     2               $C10E SPARE

* START OF CODE AT $C110
DIVIDE  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
        LDD     NN
        STD     F1              F1 = N UNTIL A NON-ZERO DIGIT
        CLR     FOUND
        CLR     RH              REMAINDER = 0
        CLR     RM
        CLR     RL
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
```

```
        LBLS    DVEXIT          NOTHING TO DO IF N <= F
        STD     CNT
*
* SET UP SOURCE BLOCK/POINTER FOR INDEX F
        LDD     FF
        LSRA                    A = F / 4096
        LSRA
        LSRA
        LSRA
        ADDA    SRC+1
        STA     SBLK
        LDD     FF
        ANDA    #$0F            OFFSET = F AND $0FFF
        ADDD    #WINDOW
        TFR     D,X             X = SOURCE POINTER
*
* SAME FOR DESTINATION
        LDD     FF
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     FF
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y             Y = DEST POINTER
*
* CHOOSE THE FAST 16-BIT PATH IF R*10+9 FITS IN 16 BITS,
* I.E. V <= 6553. OTHERWISE USE THE GENERAL 24-BIT PATH.
        LDD     VV
        CMPD    #6553
        BHI     DVLOOP
        ASLB                    V5 = 5 * V
        ROLA
        ASLB
        ROLA
        ADDD    VV
        STD     V5
        LBRA    FAST
*
* GENERAL 24-BIT PATH - ONE DIGIT PER PASS
DVLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X+             FETCH DIGIT
        STB     DIG
        CMPX    #WINEND         OFF THE END OF THE WINDOW?
        BNE     DV1
        LDX     #WINDOW         YES - NEXT BLOCK
        INC     SBLK
*
* R = R * 10 + DIGIT (24 BIT)
DV1     ASL     RL              R = R * 2
        ROL     RM
        ROL     RH
        LDD     RM              T = R * 2
        STD     TM
        LDA     RH
        STA     TH
        ASL     RL              R = R * 8
        ROL     RM
        ROL     RH
        ASL     RL
        ROL     RM
        ROL     RH
        LDD     RM              R = R*8 + R*2
        ADDD    TM
        STD     RM
        LDA     RH
        ADCA    TH
        STA     RH
        LDD     RM              R = R + DIGIT
        ADDB    DIG
        ADCA    #0
        STD     RM
        LDA     RH
        ADCA    #0
        STA     RH
```

```
*
* Q = R / V BY REPEATED SUBTRACTION (AT MOST 9 TIMES)
        CLR     QQ
DV2     LDD     RM
        SUBD    VV
        STD     TM
        LDA     RH
        SBCA    #0
        BCS     DV3             BORROW - R < V, Q IS DONE
        STA     RH
        LDD     TM
        STD     RM
        INC     QQ
        BRA     DV2
*
* RECORD THE FIRST NON-ZERO QUOTIENT DIGIT
DV3     TST     QQ
        BEQ     DV4
        TST     FOUND
        BNE     DV4
        LDD     NN              INDEX = N - CNT
        SUBD    CNT
        STD     F1
        INC     FOUND
*
* STORE THE QUOTIENT DIGIT
DV4     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     QQ
        STB     ,Y+
        CMPY    #WINEND
        BNE     DV5
        LDY     #WINDOW
        INC     DBLK
*
DV5     LDD     CNT             COUNT DOWN
        SUBD    #1
        STD     CNT
        LBNE    DVLOOP
*
DVEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK FOR BASIC
        STA     PAGREG
        LDD     F1
        STD     RES
        STD     USRARG          ALSO RETURN IT AS THE USR VALUE
        PULS    D,X,Y,U,PC      RESTORE REGISTERS AND RETURN
*
* FAST 16-BIT PATH (V <= 6553)
* D = REMAINDER R, U = QUOTIENT DIGIT, X/Y = SRC/DEST POINTERS
FAST    LDD     #0              R = 0
FLOOP   ASLB                    R = R * 2
        ROLA
        STD     TT2
        ASLB                    R = R * 8
        ROLA
        ASLB
        ROLA
        ADDD    TT2             R = R * 10
        STD     TT2
        LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X+             FETCH DIGIT
        CLRA
        ADDD    TT2             R = R * 10 + DIGIT
        CMPX    #WINEND
        BNE     FL1
        LDX     #WINDOW
        INC     SBLK
*
* QUOTIENT: TRY 5V FIRST, THEN SUBTRACT V UNTIL IT GOES NEGATIVE
FL1     SUBD    V5
        BCC     FL2
        ADDD    V5              LESS THAN 5V - UNDO, Q STARTS AT 0
        LDU     #0
        BRA     FL3
FL2     LDU     #5              AT LEAST 5V - Q STARTS AT 5
FL3     SUBD    VV
        BCS     FL4
        LEAU    1,U
```

```
        BRA     FL3
FL4     ADDD    VV              UNDO LAST SUBTRACT - D = REMAINDER
        STD     RR2
*
* RECORD THE FIRST NON-ZERO QUOTIENT DIGIT
        TST     FOUND
        BNE     FL5
        CMPU    #0
        BEQ     FL5
        LDD     NN              INDEX = N - CNT
        SUBD    CNT
        STD     F1
        INC     FOUND
*
* STORE THE QUOTIENT DIGIT
FL5     LDA     DBLK
        STA     PAGREG
        TFR     U,D
        STB     ,Y+
        CMPY    #WINEND
        BNE     FL6
        LDY     #WINDOW
        INC     DBLK
*
FL6     LDD     CNT
        SUBD    #1
        STD     CNT
        LBEQ    DVEXIT
        LDD     RR2             RESTORE REMAINDER
        LBRA    FLOOP

* WORKING STORAGE
CNT     RMB     2               DIGITS LEFT TO DO
F1      RMB     2               FIRST NON-ZERO QUOTIENT INDEX
SBLK    RMB     1               CURRENT SOURCE BLOCK
DBLK    RMB     1               CURRENT DEST BLOCK
DIG     RMB     1               DIGIT JUST FETCHED
QQ      RMB     1               QUOTIENT DIGIT
FOUND   RMB     1               NON-ZERO DIGIT SEEN FLAG
RH      RMB     1               24-BIT REMAINDER
RM      RMB     1
RL      RMB     1
TH      RMB     1               24-BIT TEMPORARY
TM      RMB     1
TL      RMB     1
CY      RMB     1               CARRY OR BORROW (0/1)
SUBF    RMB     1               SUBTRACT FLAG
IDX     RMB     2               WORKING DIGIT INDEX
V5      RMB     2               5 * V FOR THE FAST PATH
TT2     RMB     2               R * 10 TEMPORARY
RR2     RMB     2               SAVED REMAINDER

*******************************************************
* ADDSUB - ADD OR SUBTRACT A DIGIT BUFFER INTO ANOTHER
*
*   DST(I) = DST(I) + SRC(I)   FOR I = N-1 DOWN TO F  (SS=0)
*   DST(I) = DST(I) - SRC(I)                          (SS=1)
*
* THEN ANY CARRY OR BORROW IS PROPAGATED INTO DST BELOW
* INDEX F, AS FAR AS IT GOES (TOWARDS INDEX 0).
* SRC MUST NOT BE THE SAME BUFFER AS DST.

        ORG     $C300

ADDSUB  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
        CLR     CY
        LDA     SS+1
        STA     SUBF
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
        LBLS    ASEXIT          NOTHING TO DO IF N <= F
        STD     CNT
        LDD     NN              START AT INDEX N-1
        SUBD    #1
        STD     IDX
*
* SOURCE BLOCK/POINTER FOR INDEX N-1
        LSRA
        LSRA
```

```
        LSRA
        LSRA
        ADDA    SRC+1
        STA     SBLK
        LDD     IDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,X
*
* DEST BLOCK/POINTER FOR INDEX N-1
        LDD     IDX
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     IDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y
*
* MAIN LOOP - ONE DIGIT PER PASS, WORKING DOWNWARDS
ASLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X
        STB     DIG
        LEAX    -1,X            STEP BACK
        CMPX    #WINDOW-1       OFF THE START OF THE WINDOW?
        BNE     AS1
        LDX     #WINEND-1       YES - PREVIOUS BLOCK
        DEC     SBLK
*
AS1     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     ,Y              B = DEST DIGIT
        TST     SUBF
        BNE     ASSUB
*
        ADDB    DIG             ADD: B = DEST + SRC + CARRY
        ADDB    CY
        CMPB    #10
        BLO     ASNC
        SUBB    #10
        LDA     #1              CARRY
        BRA     AS2
*
ASSUB   SUBB    DIG             SUBTRACT: B = DEST - SRC - BORROW
        SUBB    CY
        BGE     ASNC
        ADDB    #10
        LDA     #1              BORROW
        BRA     AS2
*
ASNC    CLRA                    NO CARRY OR BORROW
AS2     STA     CY
        STB     ,Y              STORE RESULT DIGIT
        LEAY    -1,Y
        CMPY    #WINDOW-1
        BNE     AS3
        LDY     #WINEND-1
        DEC     DBLK
*
AS3     LDD     CNT
        SUBD    #1
        STD     CNT
        BNE     ASLOOP
*
* PROPAGATE CARRY/BORROW INTO DIGITS F-1 DOWN TO 0
* Y AND DBLK ALREADY POINT AT INDEX F-1
        TST     CY
        BEQ     ASEXIT
        LDD     FF
        BEQ     ASEXIT          NOTHING ABOVE INDEX 0
        STD     CNT
*
PRLOOP  LDA     DBLK
        STA     PAGREG
        LDB     ,Y
```

```
        TST     SUBF
        BNE     PRSUB
        INCB                    ADD CARRY
        CMPB    #10
        BLO     PRLAST
        CLRB                    9+1 = 0, CARRY ON
        BRA     PRNEXT
PRSUB   DECB                    SUBTRACT BORROW
        BPL     PRLAST
        LDB     #9              0-1 = 9, BORROW ON
PRNEXT  STB     ,Y
        LEAY    -1,Y
        CMPY    #WINDOW-1
        BNE     PR1
        LDY     #WINEND-1
        DEC     DBLK
PR1     LDD     CNT
        SUBD    #1
        STD     CNT
        BNE     PRLOOP
        BRA     ASEXIT          RAN OFF THE TOP - IGNORE
*
PRLAST  STB     ,Y              CARRY/BORROW ABSORBED
*
ASEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK FOR BASIC
        STA     PAGREG
        PULS    D,X,Y,U,PC

        END
```

Appendix E — Version 5: PI.CMD

Files: PI.ASM

Performance and layout: 1,000 digits: 2 min 20 s (measured), ≈ 450× version 1. Assemble with a09 -FPI.CMD pi.asm . Loads at $0100; usage PI 1000, or PI to be prompted.

PI.ASM

```
* PI - CALCULATE PI TO MANY DIGITS BY MACHIN'S FORMULA
*
*   PI/4 = 4*ARCTAN(1/5) - ARCTAN(1/239)
*
* FLEX 9 COMMAND. USAGE:  PI 1000   OR JUST  PI  TO BE ASKED.
* UP TO 65529 DIGITS.
*
* ONE DECIMAL DIGIT PER BYTE, HELD IN PAGED RAM: FOUR BUFFERS
* OF 16 4K BLOCKS EACH, SEEN THROUGH A 4K WINDOW AT $E000.
* THE BLOCK IS SELECTED BY WRITING $F280 (WRITE ONLY - A READ
* LATCHES THE DATA BUS INTO THE PAGE REGISTER). BLOCK 14, THE
* NATIVE $E000 RAM, IS MAPPED BACK IN BEFORE EVERY FLEX CALL.

* FLEX ENTRY POINTS
WARMS   EQU     $CD03           RETURN TO FLEX
PUTCHR  EQU     $CD18           OUTPUT CHARACTER IN A
INBUFF  EQU     $CD1B           INPUT A LINE TO THE LINE BUFFER
PSTRNG  EQU     $CD1E           CR/LF THEN STRING AT X TO $04
PCRLF   EQU     $CD24           OUTPUT CR/LF
LBPTR   EQU     $CC14           LINE BUFFER POINTER

* HARDWARE
PAGREG  EQU     $F280           PAGE SELECT REGISTER (WRITE ONLY)
WINDOW  EQU     $E000           START OF 4K WINDOW
WINEND  EQU     $F000           FIRST ADDRESS PAST THE WINDOW
NATIVE  EQU     14              BLOCK THAT IS NORMALLY AT $E000

* DIGIT BUFFERS (FIRST BLOCK OF EACH, 16 BLOCKS = 65536 DIGITS)
PBUF    EQU     16              POWER TERM 1/X^(2M+1)
TBUF    EQU     32              CURRENT TERM P/(2M+1)
ABUF    EQU     48              ARCTAN(1/5), THEN THE RESULT
BBUF    EQU     64              ARCTAN(1/239)

GUARD   EQU     6               EXTRA DIGITS TO ABSORB ROUNDING
MAXDIG  EQU     65535-GUARD     LARGEST DIGIT COUNT ALLOWED
PERLIN  EQU     50              DIGITS PER OUTPUT LINE

        ORG     $0100

******************************************************
* MAIN PROGRAM
START   LDX     LBPTR           LOOK FOR A NUMBER AFTER "PI"
        JSR     GETNUM
        BCC     GOTN
ASK     LDX     #PROMPT
        JSR     PSTRNG
        JSR     INBUFF
        LDX     LBPTR
        JSR     GETNUM
        BCS     ASK
GOTN    STD     N1              DIGITS WANTED
        BEQ     ASK
        ADDD    #GUARD          N = N1 + GUARD
        STD     NN
*
* ARCTAN(1/5), STARTING FROM 1/5 = 0.2
        LDX     #MSG5
        JSR     PSTRNG
        LDA     #ABUF
        JSR     CLEAR
        LDA     #PBUF
        JSR     CLEAR
        LDA     #PBUF           P(0) = 2
        STA     PAGREG
        LDA     #2
        STA     WINDOW
        LDD     #25
```

```
        STD     X2
        LDA     #ABUF
        STA     ZBUF
        JSR     ARCTAN
*
* ARCTAN(1/239), 1/239 = 0.0041841 REPEATING
        LDX     #MSG239
        JSR     PSTRNG
        LDA     #BBUF
        JSR     CLEAR
        JSR     FILL239
        LDD     #57121
        STD     X2
        LDA     #BBUF
        STA     ZBUF
        JSR     ARCTAN
*
* PI = 4 * (4*ARCTAN(1/5) - ARCTAN(1/239))
        LDA     #ABUF
        LDB     #4
        JSR     MULSM
        LDD     #BBUF
        STD     SRC
        LDD     #ABUF
        STD     DST
        LDD     #0
        STD     FF
        LDD     #1              SUBTRACT
        STD     SS
        JSR     ADDSUB
        LDA     #ABUF
        LDB     #4
        JSR     MULSM           CARRY OUT IS THE INTEGER PART, 3
        JSR     PCRLF
        JSR     PRINTPI
        JSR     PCRLF
        JMP     WARMS

*****************************************************
* GETNUM - READ A DECIMAL NUMBER FROM THE STRING AT X
* RETURNS D = NUMBER, CARRY CLEAR IF OK.
* CARRY SET IF NO DIGITS OR THE NUMBER IS TOO BIG.
GETNUM  LDA     ,X+             SKIP SPACES
        CMPA    #$20
        BEQ     GETNUM
        LEAX    -1,X
        CLR     GNCNT
        LDD     #0
        STD     GNVAL
GN1     LDB     ,X+
        SUBB    #'0
        BLO     GN2             NOT A DIGIT - END OF NUMBER
        CMPB    #9
        BHI     GN2
        STB     GNDIG
        INC     GNCNT
        LDD     GNVAL           VALUE = VALUE * 10 + DIGIT
        ASLB
        ROLA
        BCS     GNBAD
        STD     GNTMP
        ASLB
        ROLA
        BCS     GNBAD
        ASLB
        ROLA
        BCS     GNBAD
        ADDD    GNTMP
        BCS     GNBAD
        ADDB    GNDIG
        ADCA    #0
        BCS     GNBAD
        STD     GNVAL
        BRA     GN1
GN2     TST     GNCNT
        BEQ     GNBAD
        LDD     GNVAL
        CMPD    #MAXDIG
        BHI     GNBAD
```

```
        ANDCC   #$FE            CLEAR CARRY - OK
        RTS
GNBAD   ORCC    #$01            SET CARRY - NO GOOD
        RTS

*****************************************************
* ARCTAN - ADD THE SERIES FOR ARCTAN(1/X) INTO BUFFER ZBUF
* ON ENTRY PBUF HOLDS 1/X AND X2 = X*X.
*   Z = P/1 - P/(3X^2) + P/(5X^4) - ...
ARCTAN  LDD     #0
        STD     MM              TERM NUMBER M
        STD     FF              FIRST NON-ZERO DIGIT OF P
        STD     SGN             0 = ADD, 1 = SUBTRACT
AT1     LDD     FF              DONE WHEN P IS ALL ZERO
        CMPD    NN
        BHS     ATDONE
*
        LDD     #PBUF           T = P / (2M+1)
        STD     SRC
        LDD     #TBUF
        STD     DST
        LDD     MM
        ASLB
        ROLA
        ADDD    #1
        STD     VV
        JSR     DIVIDE
*
        LDD     #TBUF           Z = Z + OR - T
        STD     SRC
        LDB     ZBUF
        CLRA
        STD     DST
        LDD     SGN
        STD     SS
        JSR     ADDSUB
*
        LDD     #PBUF           P = P / X2, IN PLACE
        STD     SRC
        STD     DST
        LDD     X2
        STD     VV
        JSR     DIVIDE
        LDD     RES             FIRST NON-ZERO DIGIT
        STD     FF
*
        LDD     MM              NEXT TERM
        ADDD    #1
        STD     MM
        LDB     SGN+1           FLIP SIGN
        EORB    #1
        STB     SGN+1
        BRA     AT1
ATDONE  RTS

*****************************************************
* CLEAR - SET THE FIRST NN DIGITS OF BUFFER A TO ZERO
CLEAR   STA     CBLK
        LDY     NN
CL1     LDA     CBLK
        STA     PAGREG
        LDX     #WINDOW
CL2     CLR     ,X+
        LEAY    -1,Y
        BEQ     CLDONE
        CMPX    #WINEND
        BNE     CL2
        INC     CBLK
        BRA     CL1
CLDONE  LDA     #NATIVE
        STA     PAGREG
        RTS

*****************************************************
* FILL239 - FILL PBUF WITH 1/239 = 0.0041841 0041841 ...
FILL239 LDA     #PBUF
        STA     CBLK
        LDY     NN
        CLRB                    B = POSITION IN THE 7-DIGIT PATTERN
```

```
FI1     LDA     CBLK
        STA     PAGREG
        LDX     #WINDOW
FI2     LDU     #PAT239
        LDA     B,U
        STA     ,X+
        INCB
        CMPB    #7
        BNE     FI3
        CLRB
FI3     LEAY    -1,Y
        BEQ     FIDONE
        CMPX    #WINEND
        BNE     FI2
        INC     CBLK
        BRA     FI1
FIDONE  LDA     #NATIVE
        STA     PAGREG
        RTS
PAT239  FCB     0,0,4,1,8,4,1

*******************************************************
* MULSM - MULTIPLY BUFFER A BY THE SMALL NUMBER IN B, IN PLACE
* WORKS FROM DIGIT NN-1 DOWN TO 0. RETURNS THE CARRY OUT
* OF THE TOP IN CARRYO.
MULSM   STA     CBLK
        STB     MULT
        CLR     CARRYO
        LDD     NN              INDEX NN-1
        SUBD    #1
        STD     MIDX
        LSRA                    BLOCK = BUFFER + INDEX/4096
        LSRA
        LSRA
        LSRA
        ADDA    CBLK
        STA     CBLK
        LDD     MIDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,X
        LDY     NN              COUNT
ML1     LDA     CBLK
        STA     PAGREG
ML2     LDA     ,X              DIGIT * MULT + CARRY
        LDB     MULT
        MUL
        ADDB    CARRYO
        CLRA                    A = NEW CARRY
ML3     CMPB    #10
        BLO     ML4
        SUBB    #10
        INCA
        BRA     ML3
ML4     STB     ,X
        STA     CARRYO
        LEAY    -1,Y
        BEQ     MLDONE
        LEAX    -1,X
        CMPX    #WINDOW-1
        BNE     ML2
        LDX     #WINEND-1
        DEC     CBLK
        BRA     ML1
MLDONE  LDA     #NATIVE
        STA     PAGREG
        RTS

*******************************************************
* PRINTPI - PRINT "3." AND THE FIRST N1 DIGITS OF ABUF,
* PERLIN DIGITS PER LINE, LATER LINES INDENTED TWO SPACES
PRINTPI LDA     CARRYO          INTEGER PART
        ADDA    #'0
        JSR     PUTCHR
        LDA     #'.
        JSR     PUTCHR
        LDA     #ABUF
        STA     CBLK
        LDX     #WINDOW
```

```
        LDY     N1
        LDB     #PERLIN
        STB     LINCNT
PP1     LDA     CBLK            FETCH NEXT DIGIT
        STA     PAGREG
        LDA     ,X+
        LDB     #NATIVE         MAP NATIVE BLOCK BEFORE CALLING FLEX
        STB     PAGREG
        ADDA    #'0
        PSHS    X,Y
        JSR     PUTCHR
        PULS    X,Y
        CMPX    #WINEND
        BNE     PP2
        LDX     #WINDOW
        INC     CBLK
PP2     LEAY    -1,Y
        BEQ     PPDONE
        DEC     LINCNT
        BNE     PP1
        LDB     #PERLIN         NEW LINE, INDENTED
        STB     LINCNT
        PSHS    X,Y
        JSR     PCRLF
        LDA     #$20
        JSR     PUTCHR
        LDA     #$20
        JSR     PUTCHR
        PULS    X,Y
        BRA     PP1
PPDONE  RTS

*****************************************************
* DIVIDE - DST(F..N-1) = SRC(F..N-1) / V, RETURNS F1 IN RES
* (SAME CODE AS THE BASIC VERSION'S MACHINE CODE ROUTINE)
DIVIDE  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
        LDD     NN
        STD     F1              F1 = N UNTIL A NON-ZERO DIGIT
        CLR     FOUND
        CLR     RH              REMAINDER = 0
        CLR     RM
        CLR     RL
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
        LBLS    DVEXIT          NOTHING TO DO IF N <= F
        STD     CNT
*
* SET UP SOURCE BLOCK/POINTER FOR INDEX F
        LDD     FF
        LSRA                    A = F / 4096
        LSRA
        LSRA
        LSRA
        ADDA    SRC+1
        STA     SBLK
        LDD     FF
        ANDA    #$0F            OFFSET = F AND $0FFF
        ADDD    #WINDOW
        TFR     D,X             X = SOURCE POINTER
*
* SAME FOR DESTINATION
        LDD     FF
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     FF
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y             Y = DEST POINTER
*
* CHOOSE THE FAST 16-BIT PATH IF R*10+9 FITS IN 16 BITS,
* I.E. V <= 6553. OTHERWISE USE THE GENERAL 24-BIT PATH.
        LDD     VV
        CMPD    #6553
        BHI     DVLOOP
        ASLB                    V5 = 5 * V
```

```
        ROLA
        ASLB
        ROLA
        ADDD    VV
        STD     V5
        LBRA    FAST
*
* GENERAL 24-BIT PATH - ONE DIGIT PER PASS
DVLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X+             FETCH DIGIT
        STB     DIG
        CMPX    #WINEND         OFF THE END OF THE WINDOW?
        BNE     DV1
        LDX     #WINDOW         YES - NEXT BLOCK
        INC     SBLK
*
* R = R * 10 + DIGIT (24 BIT)
DV1     ASL     RL              R = R * 2
        ROL     RM
        ROL     RH
        LDD     RM              T = R * 2
        STD     TM
        LDA     RH
        STA     TH
        ASL     RL              R = R * 8
        ROL     RM
        ROL     RH
        ASL     RL
        ROL     RM
        ROL     RH
        LDD     RM              R = R*8 + R*2
        ADDD    TM
        STD     RM
        LDA     RH
        ADCA    TH
        STA     RH
        LDD     RM              R = R + DIGIT
        ADDB    DIG
        ADCA    #0
        STD     RM
        LDA     RH
        ADCA    #0
        STA     RH
*
* Q = R / V BY REPEATED SUBTRACTION (AT MOST 9 TIMES)
        CLR     QQ
DV2     LDD     RM
        SUBD    VV
        STD     TM
        LDA     RH
        SBCA    #0
        BCS     DV3             BORROW - R < V, Q IS DONE
        STA     RH
        LDD     TM
        STD     RM
        INC     QQ
        BRA     DV2
*
* RECORD THE FIRST NON-ZERO QUOTIENT DIGIT
DV3     TST     QQ
        BEQ     DV4
        TST     FOUND
        BNE     DV4
        LDD     NN              INDEX = N - CNT
        SUBD    CNT
        STD     F1
        INC     FOUND
*
* STORE THE QUOTIENT DIGIT
DV4     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     QQ
        STB     ,Y+
        CMPY    #WINEND
        BNE     DV5
        LDY     #WINDOW
        INC     DBLK
*
```

```
DV5     LDD     CNT             COUNT DOWN
        SUBD    #1
        STD     CNT
        LBNE    DVLOOP
*
DVEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK
        STA     PAGREG
        LDD     F1
        STD     RES
        PULS    D,X,Y,U,PC      RESTORE REGISTERS AND RETURN
*
* FAST 16-BIT PATH (V <= 6553)
* D = REMAINDER R, U = QUOTIENT DIGIT, X/Y = SRC/DEST POINTERS
FAST    LDD     #0              R = 0
FLOOP   ASLB                    R = R * 2
        ROLA
        STD     TT2
        ASLB                    R = R * 8
        ROLA
        ASLB
        ROLA
        ADDD    TT2             R = R * 10
        STD     TT2
        LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X+             FETCH DIGIT
        CLRA
        ADDD    TT2             R = R * 10 + DIGIT
        CMPX    #WINEND
        BNE     FL1
        LDX     #WINDOW
        INC     SBLK
*
* QUOTIENT: TRY 5V FIRST, THEN SUBTRACT V UNTIL IT GOES NEGATIVE
FL1     SUBD    V5
        BCC     FL2
        ADDD    V5              LESS THAN 5V - UNDO, Q STARTS AT 0
        LDU     #0
        BRA     FL3
FL2     LDU     #5              AT LEAST 5V - Q STARTS AT 5
FL3     SUBD    VV
        BCS     FL4
        LEAU    1,U
        BRA     FL3
FL4     ADDD    VV              UNDO LAST SUBTRACT - D = REMAINDER
        STD     RR2
*
* RECORD THE FIRST NON-ZERO QUOTIENT DIGIT
        TST     FOUND
        BNE     FL5
        CMPU    #0
        BEQ     FL5
        LDD     NN              INDEX = N - CNT
        SUBD    CNT
        STD     F1
        INC     FOUND
*
* STORE THE QUOTIENT DIGIT
FL5     LDA     DBLK
        STA     PAGREG
        TFR     U,D
        STB     ,Y+
        CMPY    #WINEND
        BNE     FL6
        LDY     #WINDOW
        INC     DBLK
*
FL6     LDD     CNT
        SUBD    #1
        STD     CNT
        LBEQ    DVEXIT
        LDD     RR2             RESTORE REMAINDER
        LBRA    FLOOP


******************************************************
* ADDSUB - DST = DST + OR - SRC FOR DIGITS N-1 DOWN TO F,
* THEN CARRY/BORROW INTO THE DIGITS ABOVE F
ADDSUB  PSHS    D,X,Y,U         KEEP BASIC'S REGISTERS
```

```
        CLR     CY
        LDA     SS+1
        STA     SUBF
        LDD     NN              CNT = N - F DIGITS TO DO
        SUBD    FF
        LBLS    ASEXIT          NOTHING TO DO IF N <= F
        STD     CNT
        LDD     NN              START AT INDEX N-1
        SUBD    #1
        STD     IDX
*
* SOURCE BLOCK/POINTER FOR INDEX N-1
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    SRC+1
        STA     SBLK
        LDD     IDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,X
*
* DEST BLOCK/POINTER FOR INDEX N-1
        LDD     IDX
        LSRA
        LSRA
        LSRA
        LSRA
        ADDA    DST+1
        STA     DBLK
        LDD     IDX
        ANDA    #$0F
        ADDD    #WINDOW
        TFR     D,Y
*
* MAIN LOOP - ONE DIGIT PER PASS, WORKING DOWNWARDS
ASLOOP  LDA     SBLK            MAP SOURCE BLOCK
        STA     PAGREG
        LDB     ,X
        STB     DIG
        LEAX    -1,X            STEP BACK
        CMPX    #WINDOW-1       OFF THE START OF THE WINDOW?
        BNE     AS1
        LDX     #WINEND-1       YES - PREVIOUS BLOCK
        DEC     SBLK
*
AS1     LDA     DBLK            MAP DEST BLOCK
        STA     PAGREG
        LDB     ,Y              B = DEST DIGIT
        TST     SUBF
        BNE     ASSUB
*
        ADDB    DIG             ADD: B = DEST + SRC + CARRY
        ADDB    CY
        CMPB    #10
        BLO     ASNC
        SUBB    #10
        LDA     #1              CARRY
        BRA     AS2
*
ASSUB   SUBB    DIG             SUBTRACT: B = DEST - SRC - BORROW
        SUBB    CY
        BGE     ASNC
        ADDB    #10
        LDA     #1              BORROW
        BRA     AS2
*
ASNC    CLRA                    NO CARRY OR BORROW
AS2     STA     CY
        STB     ,Y              STORE RESULT DIGIT
        LEAY    -1,Y
        CMPY    #WINDOW-1
        BNE     AS3
        LDY     #WINEND-1
        DEC     DBLK
*
AS3     LDD     CNT
        SUBD    #1
```

```
        STD     CNT
        BNE     ASLOOP
*
* PROPAGATE CARRY/BORROW INTO DIGITS F-1 DOWN TO 0
* Y AND DBLK ALREADY POINT AT INDEX F-1
        TST     CY
        BEQ     ASEXIT
        LDD     FF
        BEQ     ASEXIT          NOTHING ABOVE INDEX 0
        STD     CNT
*
PRLOOP  LDA     DBLK
        STA     PAGREG
        LDB     ,Y
        TST     SUBF
        BNE     PRSUB
        INCB                    ADD CARRY
        CMPB    #10
        BLO     PRLAST
        CLRB                    9+1 = 0, CARRY ON
        BRA     PRNEXT
PRSUB   DECB                    SUBTRACT BORROW
        BPL     PRLAST
        LDB     #9              0-1 = 9, BORROW ON
PRNEXT  STB     ,Y
        LEAY    -1,Y
        CMPY    #WINDOW-1
        BNE     PR1
        LDY     #WINEND-1
        DEC     DBLK
PR1     LDD     CNT
        SUBD    #1
        STD     CNT
        BNE     PRLOOP
        BRA     ASEXIT          RAN OFF THE TOP - IGNORE
*
PRLAST  STB     ,Y              CARRY/BORROW ABSORBED
*
ASEXIT  LDA     #NATIVE         RESTORE NATIVE BLOCK
        STA     PAGREG
        PULS    D,X,Y,U,PC

******************************************************
* MESSAGES
PROMPT  FCC     'NUMBER OF DIGITS? '
        FCB     4
MSG5    FCC     'ARCTAN(1/5)'
        FCB     4
MSG239  FCC     'ARCTAN(1/239)'
        FCB     4

******************************************************
* PARAMETERS FOR DIVIDE AND ADDSUB
SRC     RMB     2               SOURCE BUFFER FIRST BLOCK
DST     RMB     2               DEST BUFFER FIRST BLOCK
NN      RMB     2               N, DIGITS IN EACH BUFFER
FF      RMB     2               F, FIRST DIGIT TO PROCESS
VV      RMB     2               V, DIVISOR
SS      RMB     2               0 = ADD, 1 = SUBTRACT
RES     RMB     2               RESULT FROM DIVIDE (F1)

* WORKING STORAGE
N1      RMB     2               DIGITS WANTED
X2      RMB     2               X*X FOR THE CURRENT SERIES
ZBUF    RMB     1               ACCUMULATOR BUFFER FOR ARCTAN
MM      RMB     2               TERM NUMBER
SGN     RMB     2               0 = ADD, 1 = SUBTRACT
CBLK    RMB     1               CURRENT BLOCK (CLEAR/FILL/MUL/PRINT)
MULT    RMB     1               MULTIPLIER FOR MULSM
CARRYO  RMB     1               CARRY OUT OF MULSM
MIDX    RMB     2               INDEX FOR MULSM
LINCNT  RMB     1               DIGITS LEFT ON THIS LINE
GNVAL   RMB     2               GETNUM VALUE
GNTMP   RMB     2
GNDIG   RMB     1
GNCNT   RMB     1
CNT     RMB     2               DIGITS LEFT TO DO
F1      RMB     2               FIRST NON-ZERO QUOTIENT INDEX
SBLK    RMB     1               CURRENT SOURCE BLOCK
```

```
DBLK    RMB     1               CURRENT DEST BLOCK
DIG     RMB     1               DIGIT JUST FETCHED
QQ      RMB     1               QUOTIENT DIGIT
FOUND   RMB     1               NON-ZERO DIGIT SEEN FLAG
RH      RMB     1               24-BIT REMAINDER
RM      RMB     1
RL      RMB     1
TH      RMB     1               24-BIT TEMPORARY
TM      RMB     1
TL      RMB     1
CY      RMB     1               CARRY OR BORROW (0/1)
SUBF    RMB     1               SUBTRACT FLAG
IDX     RMB     2               WORKING DIGIT INDEX
V5      RMB     2               5 * V FOR THE FAST PATH
TT2     RMB     2               R * 10 TEMPORARY
RR2     RMB     2               SAVED REMAINDER

        END     START
```